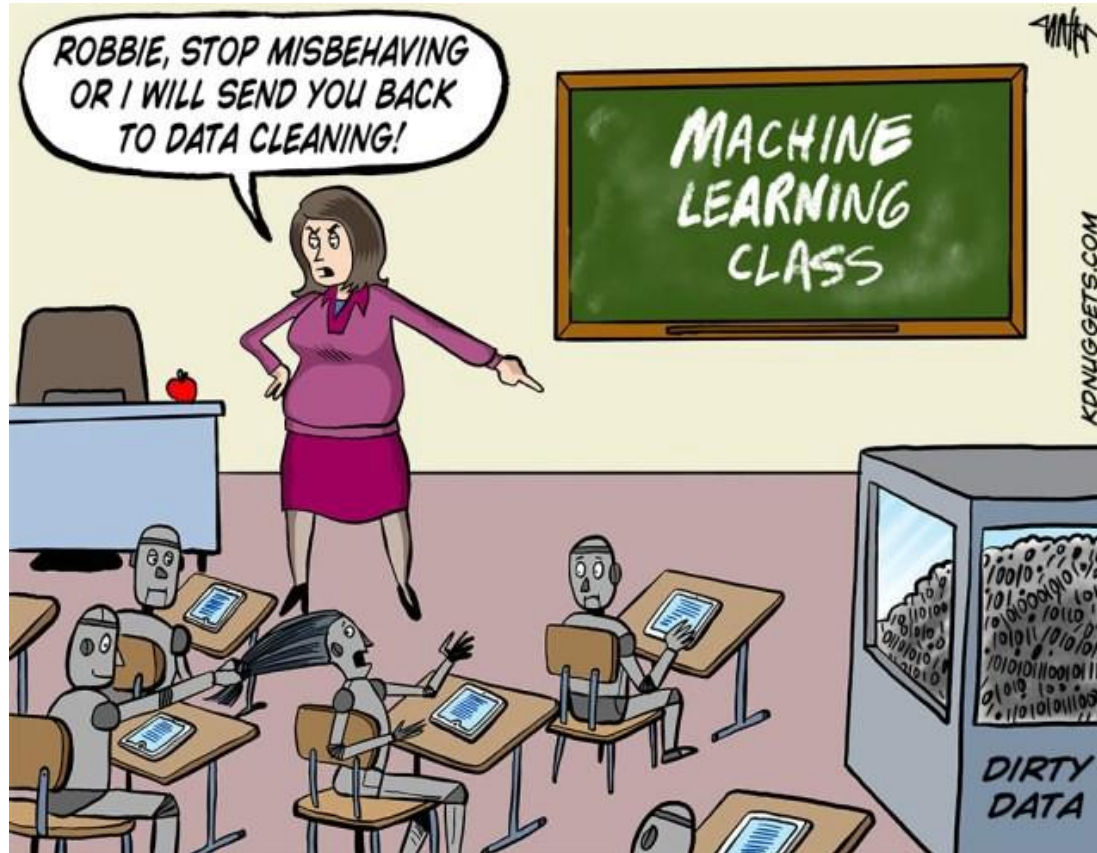


Dimensionality reduction

Machine learning

Doc. Ing. Radim Kolar, Ph.D.
Department of Biomedical Engineering,
FEEC, Brno University of Technology

Introduction



Motivation - why dimensionality reduction

Today's applications generate a lot of data... and **features**. We can find application with only few features...and also with several **thousands** of features.

It's usually not feasible to analyze each and every variable at a microscopic level.

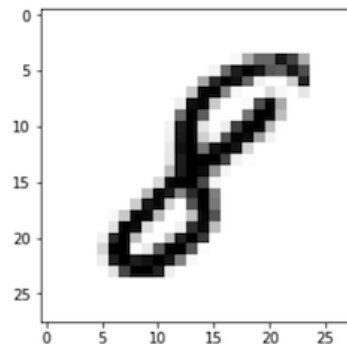
We need some mathematical tools to assess feature relevance, correspondence, validity...

There are many reasons to reduce the number of features to sufficient minimum, e.g. **computation complexity, generalization ability of the classifier** (for limited number of training patterns N , keeping the number of features as small as possible is in the line with our aim to design classifier with good generalization ability; see Curse of dimensionality)

Example

Consider images of digits with size 28x28 pixels.

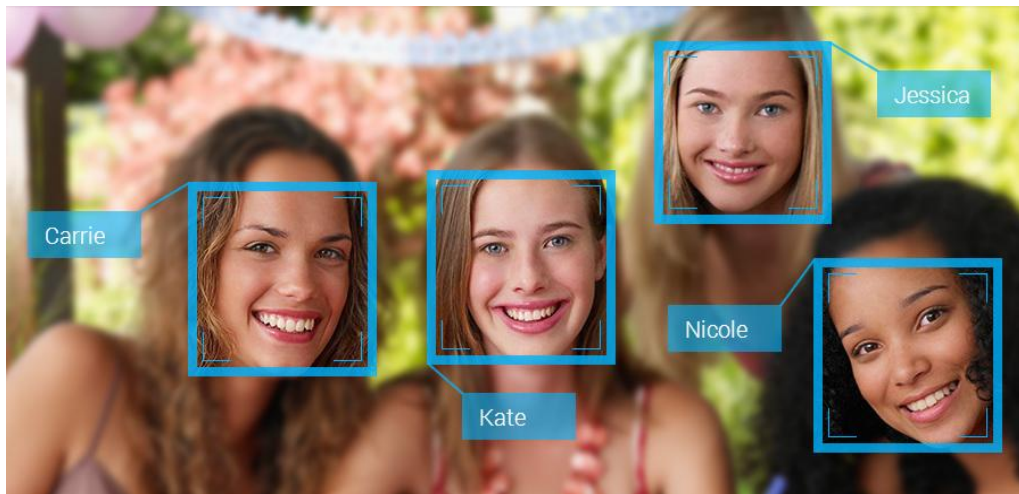
Representation of this digit image as a vector will result in $28 \times 28 = 784$ size of vector.



Example

Face recognition is largely used in many application, e.g. security, smartphones, biometry...

The RGB values of particular pixel is not very informative and useful for recognition. Rather the **high-level features** are much better (shapes, distance...).



https://miro.medium.com/max/896/1*MnbUSSXG1IDS9mXC0H8cKQ.png

Dimensionality reduction

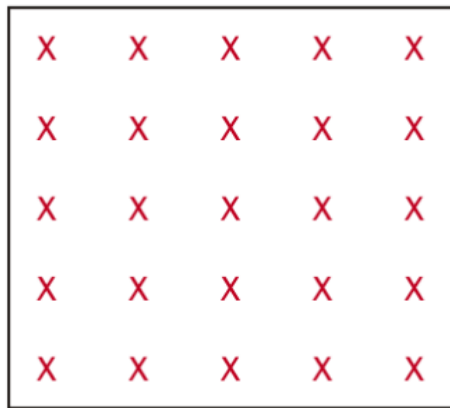
- Don't use *automatically* the tools for data analysis. It preferable to think and get know more about the features before analysis and explore data first, before building the model!
- Check feature redundancy.
- Improves the classification performance.
- Less dimensions lead to less computation/training time.
- Helps in visualizing data.
- Helps in data compression and reduced storage space.

Curse of Dimensionality

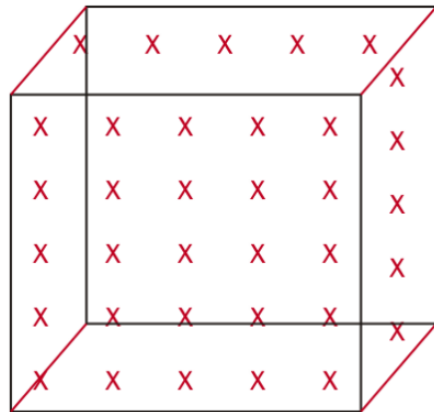
As the number of features grows, we need more data to reasonably span the N-dimensional space (i.e. to keep the same distance between samples). The number of samples grows exponentially - the example below: 5^1 , 5^2 , 5^3 ...



A one-dimensional features space
with five data points



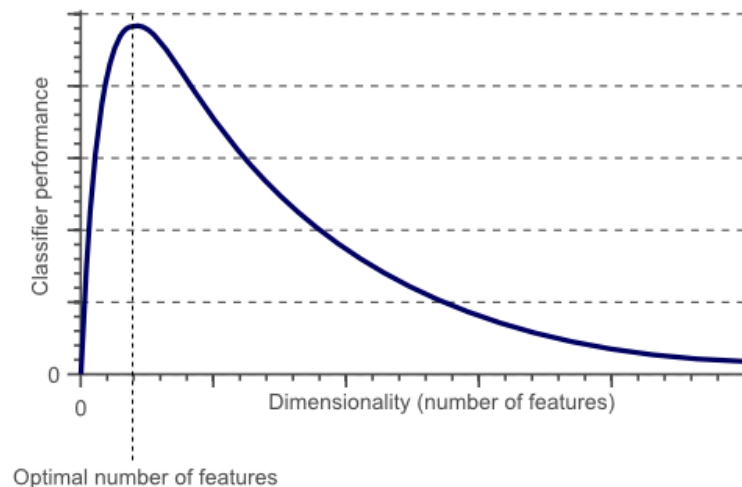
A two-dimensional features
space with 25 data points



A three-dimensional features space
with 125 data points

Curse of dimensionality - Hughes Phenomenon

Hughes phenomenon states that when you have a dataset of fixed number of samples, let's say 1000, then the more dimensions the samples have, the worse the system will be. For example if we have 2 dimensional feature vector, we will be able to train a decent predicting system with around 1000 samples, but when we have 500 dimensional samples, then 1000 samples won't be enough to train a good predicting system. This is also known as **curse of dimensionality**.



https://miro.medium.com/max/1020/0*iC_4D-LTTRUiZJeM.png

Curse of dimensionality - Hughes Phenomenon

The intuitive explanation is that to train a good predicting system it has to "see" a lot of possible combinations of possible values in order **to learn** the patterns in the data. With more dimensions there is more possible combinations and so you need many more samples.

Dimensionality reduction methods

These methods can be classified from different point of views. For example as **linear** approaches (PCA, LDA) or **nonlinear** (Kernel PCA, t-SNE).

If we think about features we can think about **features selection** (filter-based and wrapper-based approaches) and about **features reduction**.

Introduction

Now, we will look on different approach for analyzing the features and methods for features reduction and selection:

1. Basic analysis of the features
2. Selected measures of dependency
3. Feature reduction methods - PCA, kernel PCA, LDA, t-SNE, autoencoders
4. Feature selection methods - random tree, backward/forward selection, MRMR

First of all, let's look on the first step you should do if you have to analyze/classify a large number of data.

1. Basic analysis of the features

Basic analysis of the features

1. **Missing data** - check if there are some missing values for each variable. Discard some variable if missing data is above some threshold (e.g. 30%).
2. **Outlier removal** - an outlier is defined as a point that lies very far from the mean of corresponding variable. It might be a source of measurement error, but not always. In that case, proper cost function should be used.
3. **Low variance** - consider an extreme case - a variable in our dataset where all the observations have the same value, say 1. If we use this variable, do you think it can improve the model we will build? No, because this variable will have zero variance. Therefore - filter out the variables with low variances.
4. **High dependency** - high dependency between two variables means they have similar trends and are likely to carry similar information. We can calculate e.g. the correlation coefficient between independent numerical variables and if it crosses a certain threshold value, we can drop one of the variables. As a general guideline, we should keep those variables which show higher correlation with the target variable.

2. (Selected) Measures of dependency

(Selected) Measures of dependency

- A. Pearson correlation coefficient
- B. Spearman correlation coefficient
- C. Distance correlation
- D. Mutual information

A. Pearson correlation coefficient

It computes the relation between normalized values of two variables X and Y, with corresponding means μ and standard deviations σ :

$$\rho = \frac{E[\{X - \mu_x\}\{Y - \mu_y\}]}{\sigma_x \sigma_y} = \frac{E[\{X - \mu_x\}]}{\sigma_x} \frac{E[\{Y - \mu_y\}]}{\sigma_y}$$

We need to subtract the mean and divide it by standard deviation due to different units in which the features are measured (seconds, meters, millimeters, mW...).

```
x = np.random.uniform(-1, 1, 100000)
print pearsonr(x, x**2)[0]
```


Remark - Data normalization

In many practical applications the features are measured within different units and ranges. Thus the features with large values may have a large influence in the cost function than features with low values, although it doesn't necessary reflect their respective significance in the design of the classifier.

A straightforward (not only one) technique is usually used. For a feature vector $\mathbf{X}$ the normalization is:

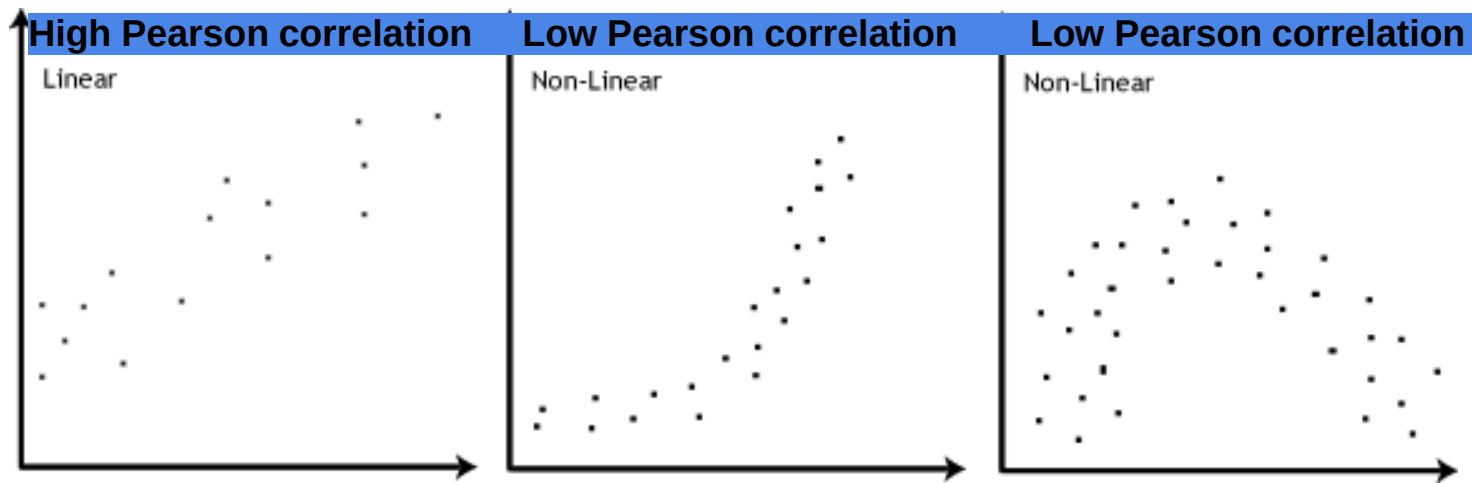
$$x_i^{norm} = \frac{x_i - \mu}{\sigma}$$

Where μ is a mean computed from feature samples and σ is corresponding standard deviation.

Other possible normalization: range 0 - 1, range +1:-1, extraction of mean only...

A. Pearson correlation

One obvious drawback of Pearson correlation as a feature ranking mechanism is that it is only **sensitive to a linear relationship**; i.e. Pearson's correlation determines the degree to which a relationship is linear. If the relation is non-linear, Pearson correlation can be close to **zero** even if there is a “*one-to-one*” (i.e. very close) correspondence between the two variables.



B. Spearman correlation coefficient

The Spearman's Rank Correlation Coefficient is the non-parametric statistical measure used to study the strength of association between the two ranked variables. This method is applied to the ordinal set of numbers, which can be arranged in order, i.e. one after the other so that ranks can be given to each. In the rank correlation coefficient method, the ranks are given to each individual on the basis of its quality or quantity, such as ranking starts from position 1st and goes till Nth position for the one ranked last in the group. The formula to calculate the rank correlation coefficient R is:

$$R = \frac{(1 - 6 \sum D^2)}{N (N^2 - 1)} = \frac{(1 - 6 \sum D^2)}{N^3 - N}$$

D - difference of ranks, N - number of observations.

Example: <https://accendoreliability.com/spearman-rank-correlation-coefficient/>

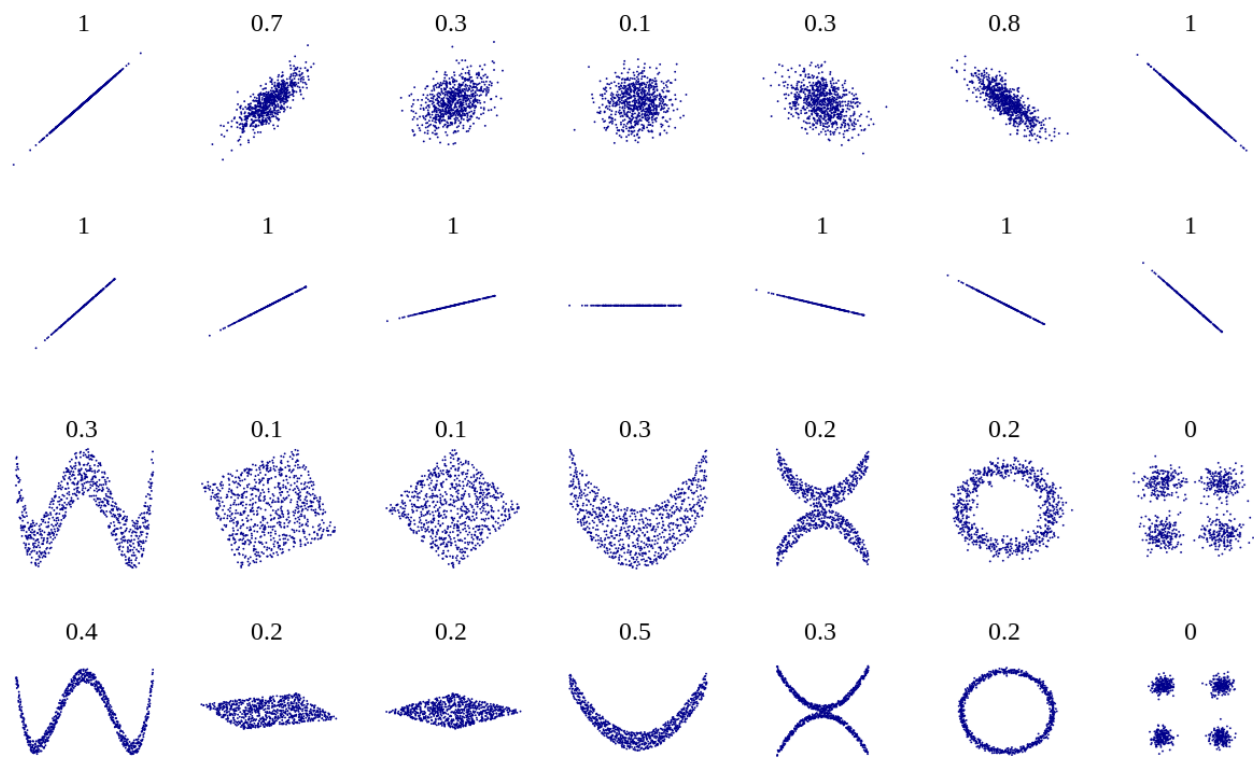
C. Distance correlation measure

Distance correlation is a measure of dependence between two random vectors $\mathbf{X}$ and $\mathbf{Y}$ (samples of features), and its value ranges from zero to one. It uses Euclidean distance in its formula, and it has the main properties:

It is zero **if and only if** the two random vectors are **independent**, which means that obtaining zero value characterizes the independence between the two vectors.

For some linear or non-linear dependence, the value is greater than 0.

Distance correlation



Distance correlation measure

Sample distance correlation coefficient can be practically computed simply. Let x_k and y_k ($k = 1, 2, \dots, N$) be samples from some measurement (e.g. we measure temperature and pressure at the same time). Therefore we get a real valued vector of random variables (X, Y) . First, compute the N by N **distance matrices** $(a_{j,k})$ and $(b_{j,k})$ containing all pairwise distances:

$$a_{j,k} = |x_j - x_k|$$
$$b_{j,k} = |y_j - y_k|$$

i.e. for each sample we compute the distance to all samples.

Now, we can compute the correlation between these matrices, but firstly we should extract the mean values.

Distance correlation measure

From each entry of distance matrices we subtract the $\bar{a}_{j\cdot}$ (the j -th row mean), $\bar{a}_{\cdot k}$ (the k -th column mean), and $\bar{a}_{\cdot\cdot}$ is the *grand* (total) mean of the distance matrix of the X sample. Similar for distance matrix b . We must add the grand mean because we subtract twice the *vector* (row, column) means:

$$A_{j,k} = a_{j,k} - \bar{a}_{j\cdot} - \bar{a}_{\cdot k} + \bar{a}_{\cdot\cdot}$$

$$B_{j,k} = b_{j,k} - \bar{b}_{j\cdot} - \bar{b}_{\cdot k} + \bar{b}_{\cdot\cdot}$$

Distance correlation measure

The **squared sample covariance** is then defined as:

$$\text{dCov}_n^2(X, Y) := \frac{1}{N^2} \sum_{j=1}^N \sum_{k=1}^N A_{j,k} B_{j,k}.$$

And finally the distance correlation measure:

$$\text{dCor}(X, Y) = \frac{\text{dCov}(X, Y)}{\sqrt{\text{dVar}(X) \text{dVar}(Y)}},$$

Where the variances (or standard deviations) are used to “normalize” the data (they are measured in different units).

Matlab Code

Generate data

```
N = 1000;  
x = randn(1,N);  
y = x.*x;  
  
x = x(:);  
y = y(:);  
  
figure(1);  
plot( x, y, 'r*' );
```

Compute A_{jk} , B_{jk}

```
a = zeros( N, N );  
b = zeros( N, N );  
for ii = 1:N  
    a(ii,:) = abs(x(ii) - x);  
end  
for ii = 1:N  
    b(ii,:) = abs(y(ii) - y);  
end  
  
RowMean = mean(a,1);  
ColumnMean = mean(a,2);  
a = bsxfun(@minus, a, RowMean );  
a = bsxfun(@minus, a, ColumnMean );  
a = a + mean( RowMean );  
  
RowMean = mean(b,1);  
ColumnMean = mean(b,2);  
b = bsxfun(@minus, b, RowMean );  
b = bsxfun(@minus, b, ColumnMean );  
b = b + mean( RowMean );
```

Compute the coefficient

```
% Distance correlation  
dcov = sum(sum(a.*b))/(N^2)  
dvarx = sum(sum(a.*a))/(N^2)  
dvary = sum(sum(b.*b))/(N^2)  
  
dcor = sqrt(dcov/sqrt(dvarx*dvary))
```

D. Mutual information

Another, more robust option for correlation estimation is mutual information. This measure is used in many areas of machine learning, data processing, image registration etc.

For in-depth information about MI and feature selection see: [Vergara at al.](#)

D. Mutual information

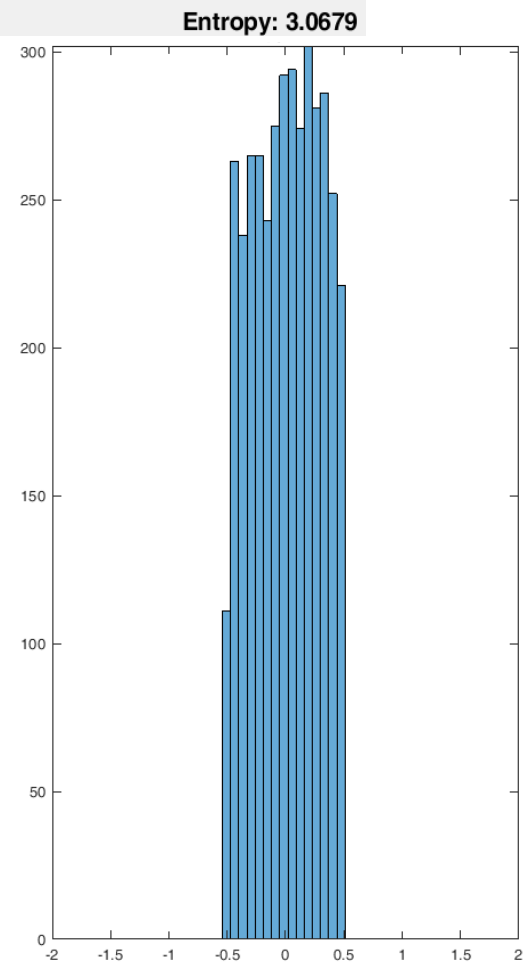
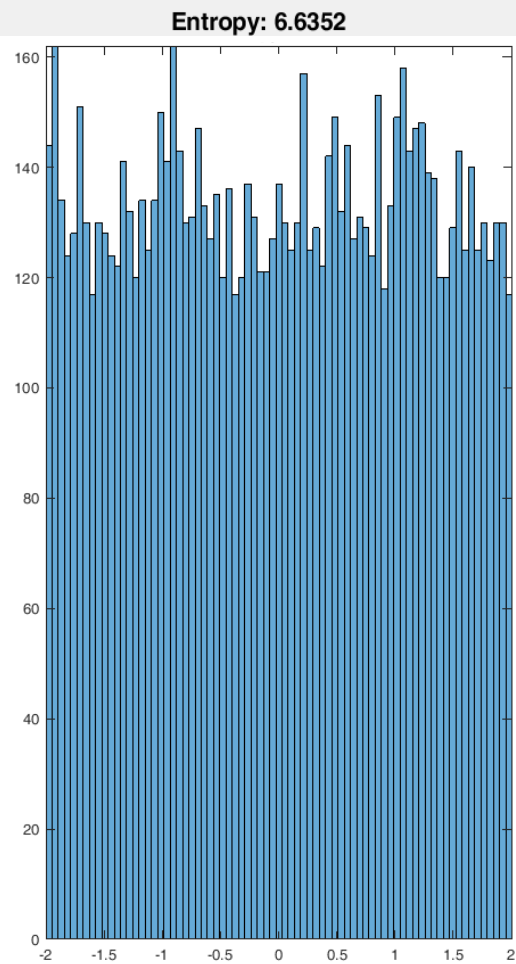
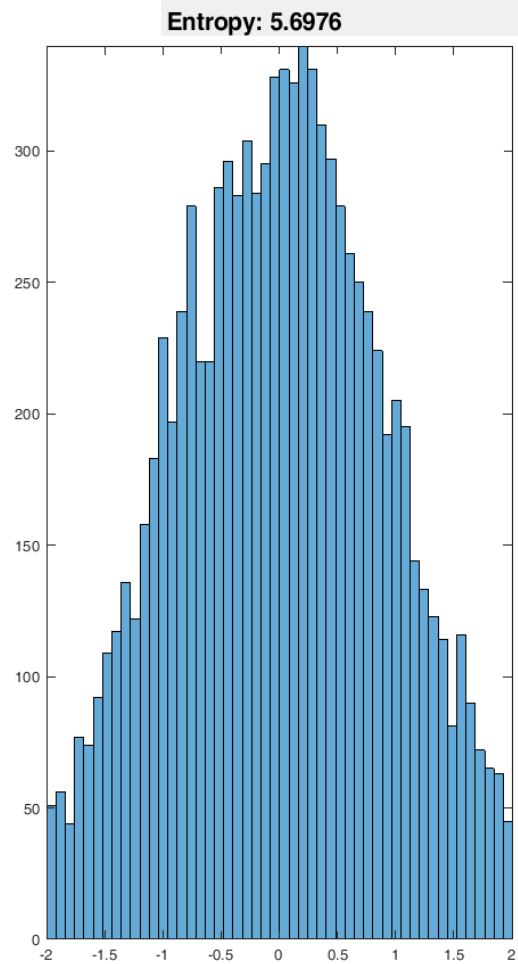
We will start with entropy. Formally, the entropy of a discrete random variable x , with probability mass function $p(x(i))=Pr\{x=x(i)\}$ is defined as:

$$H(x) = - \sum_{i=1}^n p(x(i)) \log_2 p(x(i))$$

Entropy (H) is a measure of uncertainty of a random variable. The uncertainty is related to the probability of occurrence of an event. Intuitively, **high entropy means that each event has about the same probability of occurrence, while low entropy means that each event has a different probability of occurrence.**

Note: A **probability mass function** differs from a **probability density function** (PDF) in that the PDF is associated with continuous rather than discrete random variables. Formally, the values of the PDF are not probabilities as such: a PDF must be integrated over an arbitrary interval to yield a probability.

Example – three different histograms



Mutual information

Let $\mathbf{x}$ and $\mathbf{y}$ be two random discrete variables. The **joint entropy** of $\mathbf{x}$ and $\mathbf{y}$, with **joint mass probability** $p(\mathbf{x}(i), \mathbf{y}(j))$, is the sum of the uncertainty contained by the two variables. Formally, joint entropy is defined as follows:

$$H(\{x, y\}) = - \sum_{i=1}^n \sum_{j=1}^n p(x(i), y(j)) \cdot \log_2(p(x(i), y(j)))$$

The joint entropy has values in the range,

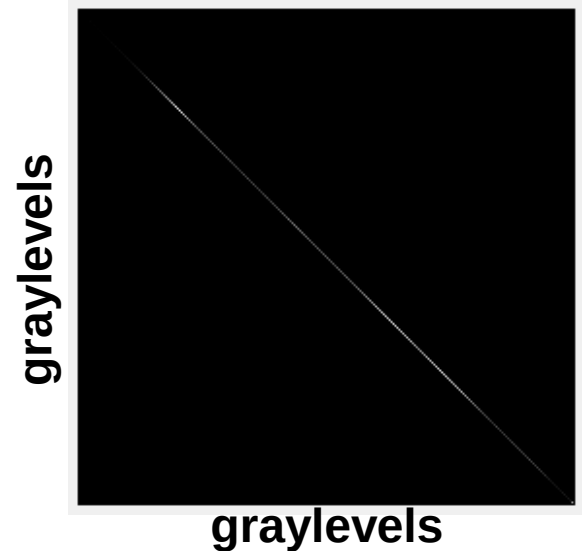
$$\max(H(x), H(y)) \leq H(\{x, y\}) \leq H(x) + H(y)$$

The maximum value of this inequality happens when x and y are completely independent. The minimum value occurs when x is completely dependent on y .

Mutual information

How a joint histogram of two completely identical processes will look like?

Consider an image as source of samples of one random process and the copy of this image as a second random process.

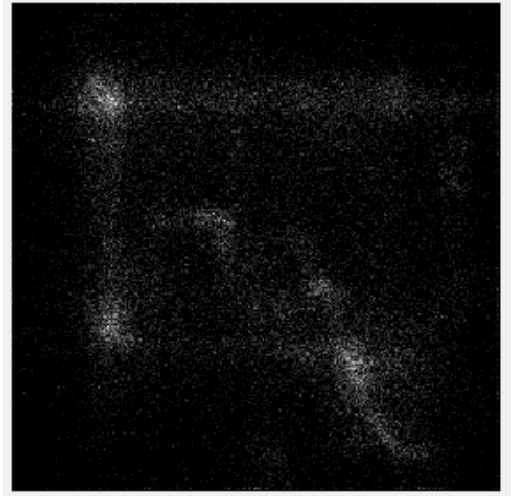


Mutual information

Consider this situation: x - the same; y - slightly scaled version of x . This means that y is 'somehow' dependent on x .



graylevels



graylevels

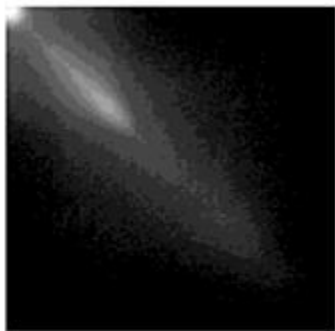
Mutual information

Influence of joint histogram on joint entropy.

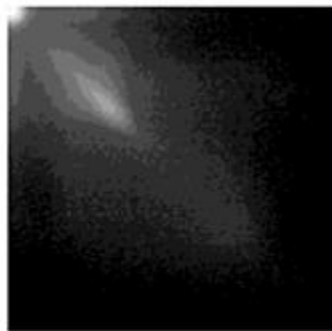
Examples:



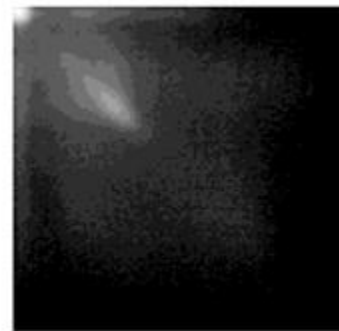
3.82



6.79



6.98



7.15

Joint entropy values below the images.

Mutual information

The mutual information $I(x,y)$ is a measure of the amount of information that one random variable has about another variable:

$$I(x; y) = \sum_{i=1}^n \sum_{j=1}^n p(x(i), y(j)) \cdot \log \left(\frac{p(x(i), y(j))}{p(x(i)) \cdot p(y(j))} \right)$$

where $I(x,y)$ is zero when x and y are statistically independent: $p(x(i),y(i)) = p(x).p(y)$. $I(x,y)$ can be expressed as:

$$I(x;y) = H(x) + H(y) - H(x,y)$$

Mutual information - problems

It can be inconvenient to use directly for feature ranking for two reasons though.

Firstly, the MI values can be incomparable between two datasets.

Secondly, it can be inconvenient to compute for continuous variables: in general the variables need to be discretized by binning, but the mutual information score can be quite sensitive to bin selection.

Maximal information coefficient is a technique developed to address these shortcomings. It searches for optimal binning and turns mutual information score into a metric that lies in range $[0;1]$.

Maximal information coefficient - MIC

[Reshef et al.](#) defined MIC as mutual information estimate $I(x;y)$ computed using a data-dependent binning scheme. Let n_x and n_y , respectively, denote the number of bins imposed on the x and y axes. The principle of the MIC binning scheme is:

- 1) The total number of bins ($n_x \cdot n_y$) does not exceed some user-specified value B ;
- 2) For specific n_x and n_y values – find the maximum MI (note, that the bin scheme is generally not equal).

3) Compute MIC for all possible binning scheme:
$$MIC(x, y) = \frac{I(x, y)}{Z_{MIC}}$$

where $Z_{MIC} = \log_2(\min(n_x, n_y))$

- 4) Take the maximum value from MIC matrix.

Feature reduction methods
(PCA, kernel PCA, LDA, t-SNE,
autoencoders)

Principle Component Analysis (PCA)

Principle Component Analysis (PCA)

PCA is one of the oldest and most widely used dimension reduction method. Its idea is simple - reduce the dimensionality of a dataset, while preserving as much *variability* as possible. It is assumed that the high variability of principal components captures most of the statistical information contained in the dataset.

The input variables are *transformed* into the new/output variables such that they are **linear functions old variables**. During this process the **variance is maximized**. New variables are also **uncorrelated** with each other.

Sources:

https://www.researchgate.net/publication/229158312_Kernel_Principal_Component_Analysis_and_its_Applications_in_FaceRecognition_and_Active_Shape_Models

Remark: Dot product, geometric definition, vector projection

Consider two vectors (**a**, **b**) and their dot product:

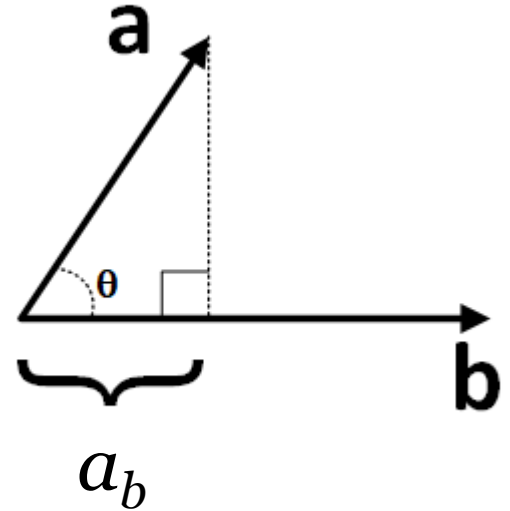
$$\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

Now consider the (scalar) projection of vector **a** onto **b** vector:

$$a_b = \|\mathbf{a}\| \cos \theta$$

Therefore, the projection and dot product will be the same if we project on unit vector:

$$\|\mathbf{b}\| = 1$$



PCA

Consider a set of points $\mathbf{x}_i$, $i = 1, \dots, n$, in an m -dimensional space such that their mean $\mu = 0$, i.e., centroid is at the origin.

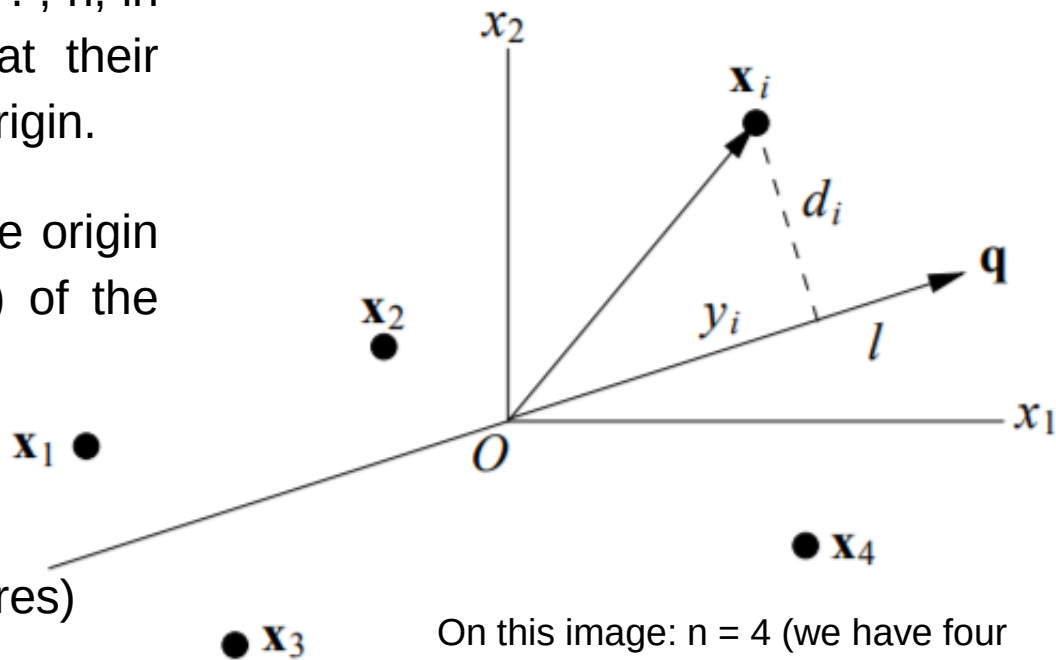
We want to find a line ℓ through the origin that maximizes the projections (y_i) of the points $\mathbf{x}_i$ on ℓ .

Recap:

n - number of points (samples)

m - dimensionality (number of features)

Note: we assume that: $\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \mathbf{0}$



On this image: $n = 4$ (we have four points) and $m = 3$ (3D space).

PCA

Let $\mathbf{q}$ denote the unit vector along line ℓ . Then, the projection y_i (i.e. a **single value** representing the distance from **origin**) of $\mathbf{x}_i$ on ℓ is:

$$y_i = \mathbf{x}_i^\top \mathbf{q} \quad \left[\quad \right] \begin{bmatrix} \quad \\ \quad \end{bmatrix}$$

You can check that this projection results in a scalar value for one sample, i.e. vector of feature values. The **mean value of the squared projection**, which is the **variance** V over all samples is:

$$V = \frac{1}{n} \sum_{i=1}^n y_i^2 = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i^\top \mathbf{q})^2 \geq 0.$$

PCA

Because $\mathbf{x}_i^\top \mathbf{q} = \mathbf{q}^\top \mathbf{x}_i$, expanding previous equation gives

$$V = \frac{1}{n} \sum_{i=1}^n (\mathbf{q}^\top \mathbf{x}_i)(\mathbf{x}_i^\top \mathbf{q}) = \mathbf{q}^\top \left[\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \right] \mathbf{q}.$$

The middle factor is the **covariance matrix $\mathbf{C}$** of the data points:

$$\mathbf{C} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top.$$

$\left[\begin{array}{c} \\ \\ \end{array} \right] \left[\right]$ Each element represents average of multiplications between particular features

Now, we want to find a **unit vector $\mathbf{q}$ (i.e. direction of line ℓ)** that maximizes the variance V , i.e.,

$$\text{maximize } V = \mathbf{q}^\top \mathbf{C} \mathbf{q} \quad \text{subject to } \|\mathbf{q}\| = 1.$$

PCA

This is a constrained optimization problem. We will use [Lagrange multiplier method](#):

Let's combine V and the constraint using Lagrange multiplier λ :

$$\text{maximize } V' = \mathbf{q}^\top \mathbf{C} \mathbf{q} - \lambda(\mathbf{q}^\top \mathbf{q} - 1).$$

Now, we differentiate V' with respect to $\mathbf{q}$ and set to 0:

$$\frac{\partial V'}{\partial \mathbf{q}} = 2\mathbf{q}^\top \mathbf{C} - 2\lambda \mathbf{q}^\top = 0.$$

PCA

$$\mathbf{C} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^{\top}.$$

Rearranging the terms gives:

$$\mathbf{q}^{\top} \mathbf{C} = \lambda \mathbf{q}^{\top}.$$

Since the **covariance matrix $\mathbf{C}$** is **symmetric**, $\mathbf{C}^{\top} = \mathbf{C}$. So, transposing both sides of previous equation gives:

$$\mathbf{C} \mathbf{q} = \lambda \mathbf{q}.$$

This equation is called an **eigenvector equation**, where $\mathbf{q}$ is the eigenvector (a vector) and λ is the **eigenvalue** (a scalar).

Thus, the eigenvector $\mathbf{q}$ of $\mathbf{C}$ gives the line ℓ that maximizes variance V .

PCA

We showed how to compute the projection only on single line ℓ and we've got a unit eigenvector $\mathbf{q}$. Let's name this (first) eigenvector as $\mathbf{q}_1$ and compute the difference between each feature vector $\mathbf{x}_i$ and projection to $\mathbf{q}_1$:

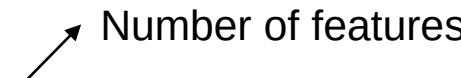
$$\mathbf{x}'_i = \mathbf{x}_i - \mathbf{x}_i^\top \mathbf{q}_1 \mathbf{q}_1$$

The second term represents the projection (scalar value) of vector $\mathbf{x}_i$ onto the eigenvector ($\mathbf{x}_i^\top \mathbf{q}_1$) and the following multiplication by (unit vector) $\mathbf{q}_1$ gives the vector... so the difference between two vectors can be calculated.

Then - repeat the method on (a new set of features) $\mathbf{x}'_i$ to get $\mathbf{q}_2$, then $\mathbf{q}_3$, $\mathbf{q}_4$...

PCA

In general, $m \times m$ covariance matrix $\mathbf{C}$ has m eigenvectors (m is dimensionality):



$$\mathbf{C} \mathbf{q}_j = \lambda_j \mathbf{q}_j, \quad j = 1, \dots, m.$$

Transpose both sides gives:

$$\mathbf{q}_j^\top \mathbf{C} = \lambda_j \mathbf{q}_j^\top, \quad j = 1, \dots, m,$$

so we get the row vectors $\mathbf{q}_j^\top$. Stack it to the matrix gives the final:

$$\begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_m^\top \end{bmatrix} \mathbf{C} = \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \lambda_m \end{bmatrix} \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_m^\top \end{bmatrix}$$

PCA

Denote matrix $\mathbf{Q}$ and Λ as

$$\mathbf{Q} = [\mathbf{q}_1 \ \cdots \ \mathbf{q}_m], \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_m).$$

Then the last equation becomes:

$$\mathbf{Q}^\top \mathbf{C} = \Lambda \mathbf{Q}^\top$$

or

$$\mathbf{C} = \mathbf{Q} \Lambda \mathbf{Q}^\top.$$

This is the **matrix equation of eigendecomposition**. Recall, that $\mathbf{C}$ is symmetric covariance matrix.

How this relates to SVD (singular value decomposition) – see nice explanation [here](#).

PCA properties

The eigenvectors are orthonormal:

$$\mathbf{q}_j^\top \mathbf{q}_j = 1,$$

$$\mathbf{q}_j^\top \mathbf{q}_k = 0, \quad \text{for } k \neq j$$

So, the eigenmatrix $\mathbf{Q}$ is orthogonal:

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{Q} \mathbf{Q}^\top = \mathbf{I}$$

The eigenvalues are arranged to be sorted in descending order (representing the described variance) $\lambda_j \geq \lambda_{j+1}$.

Value λ_j is equal to the variance σ_j^2 of data along $\mathbf{q}_j$.

PCA properties

Each entry of particular eigenvector (*loadings*) tells us how much the corresponding feature contribute to the new component.

Projection of each sample onto the eigenvectors provides numbers (*score*), which defines the coordinate of this sample in the new space.

$\mathbf{q}_1$ gives orientation of the largest variance.

$\mathbf{q}_2$ gives orientation of largest variance orthogonal to $\mathbf{q}_1$ (2nd largest variance).

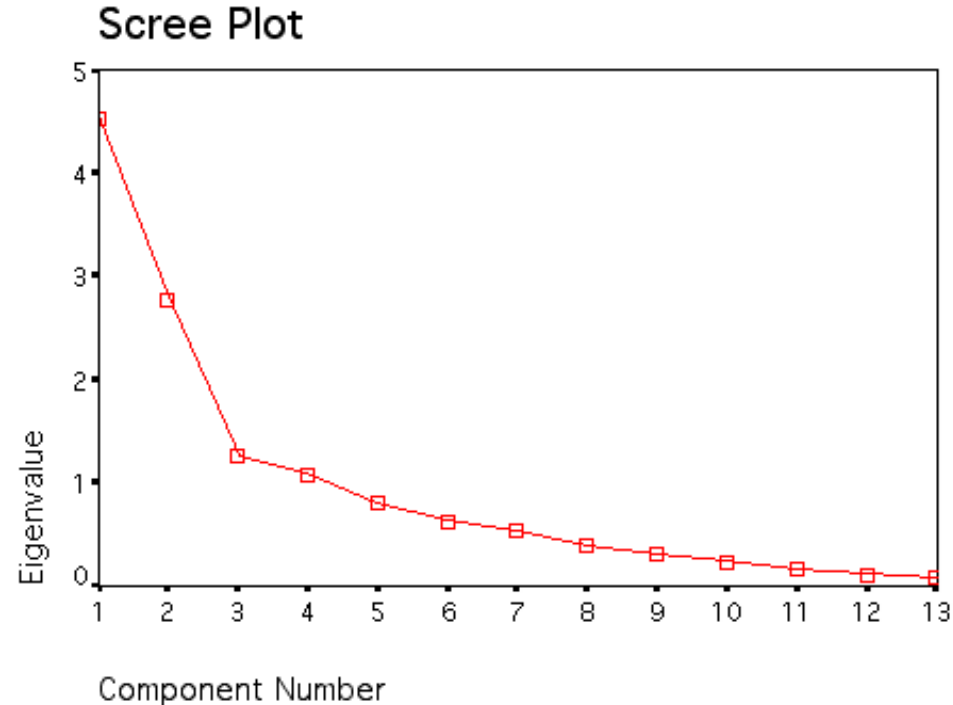
$\mathbf{q}_j$ gives orientation of largest variance orthogonal to $\mathbf{q}_1, \dots, \mathbf{q}_{j-1}$ (j-th largest variance). $\mathbf{q}_m$ is orthogonal to all other eigenvectors (least variance)

PCA - Scree plot

A scree plot is a *diagnostic tool* to check whether PCA works well on your data or not. Principal components are created in order of the amount of variation they cover: PC1 captures the most variation, PC2 — the second most, and so on.

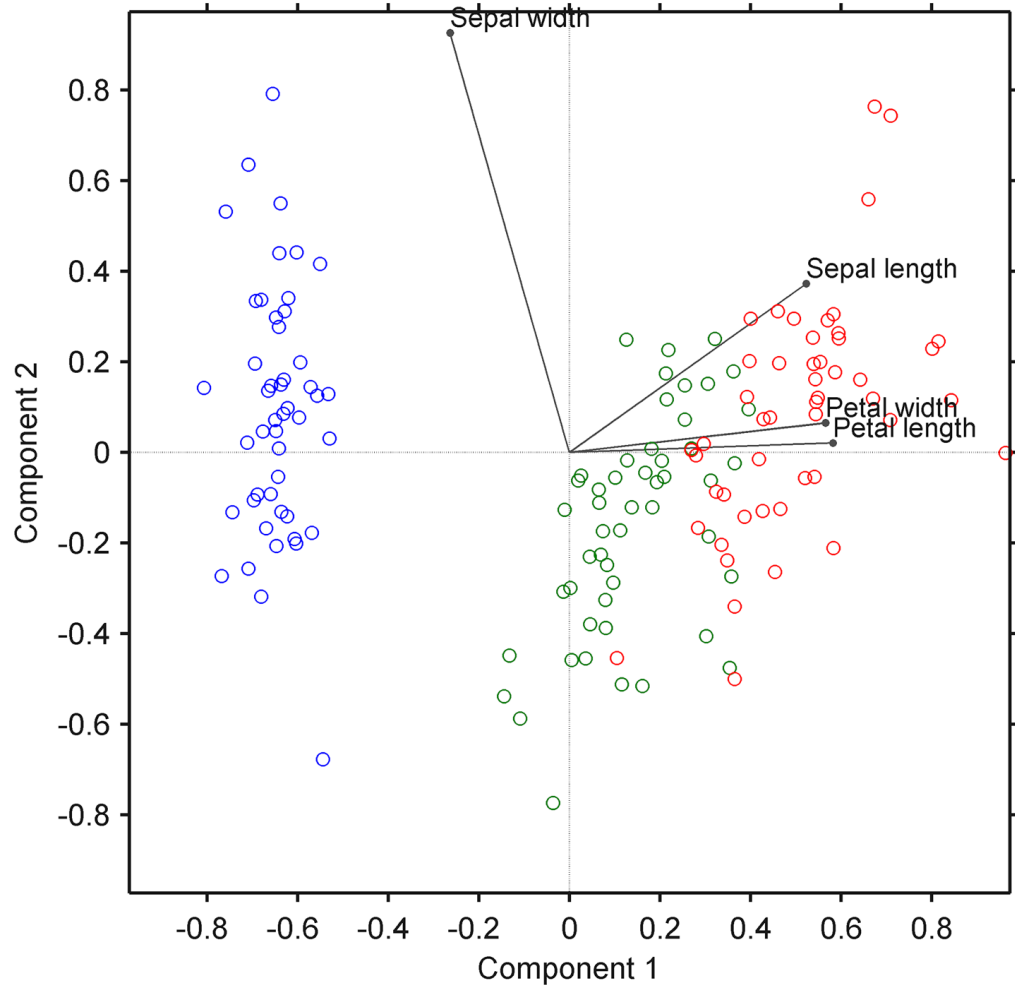
The image shows 13 PCs (there were 13 features). The 1st and 2nd covers most of the variability in the data.

Eigenvalues can be converted to %.



PCA biplot/loading plot

PCA analysis of [Iris flower dataset](#) showing how different features (lines named Sepal width etc.) contribute to 1st and 2nd principal component (axis). Three features on the right contribute almost equally to PC1 while the Sepal width mainly contributes to PC2. The angle between the lines represents “how much the features are correlated”. Angle 90° means “no correlation”. The color dots represent each sample projected into the new (principal component) space. Three colors - three different classes.



PCA limitation

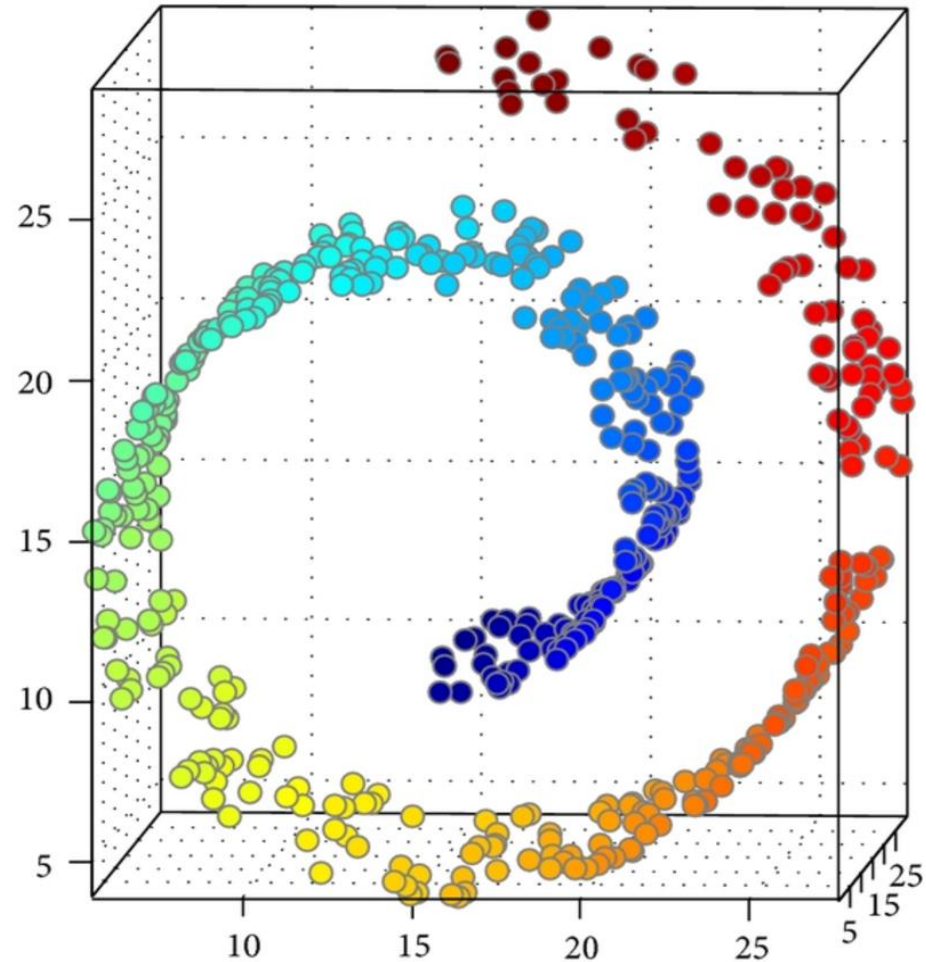
- PCA is a **linear projection method** and it could fail (e.g. lead you to wrong conclusions) on some datasets;
- variance maximization sometimes does not lead to the desired outcome;
- the results of PCA depend on the scaling of features;
- it could be difficult to provide physically meaningful interpretation of generated projections;
- the orthogonality of projection directions can be restrictive as the informative dimensions do not necessarily have to be orthogonal to each other;
- it is potentially computationally demanding to compute (for large dimensionality);
- it doesn't consider labels.

Kernel PCA

Kernel PCA

In some (real) cases the data (i.e. set of features) is not appropriate for PCA. If we apply standard PCA for the data shown in the figure, it will fail to find good representative direction. Kernel PCA (KPCA) rectifies this limitation.

Kernel PCA just performs PCA in a new space. It uses so called *Kernel trick* to find principal components in different space (possibly high dimensional space).

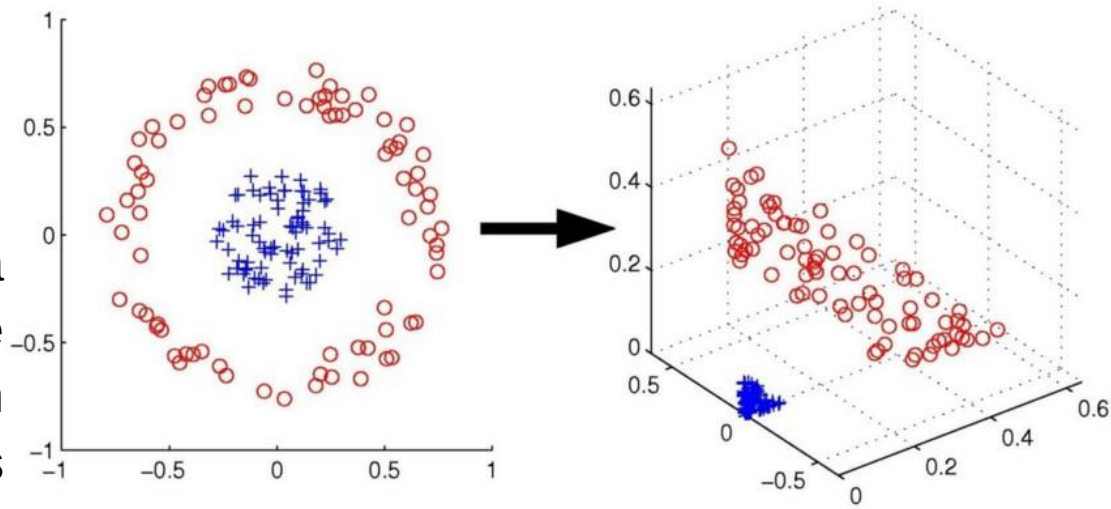


Kernel PCA

In the example, the original data is in 2-dimension. Suppose we denote it as, $\mathbf{x}=\{x_1, x_2\}$. We can see that $\mathbf{x}$ is inseparable in its space. But they are separable in a transformed space (right plot) given by,

$$\Phi(\mathbf{x}) \rightarrow x_1^2, x_2^2, \sqrt{2}x_1x_2$$

where, Φ is a transform function from 2-D to 3-D applied on $\mathbf{x}$.



https://www.researchgate.net/figure/From-a-to-d-a-Swiss-roll-and-its-hash-codes-embedded-to-3D-by-PCA-after-applying_fig3_258392870

In general, we map original data from m dimensional space to p dimensional space, $m < p$. And we hope in *better* representation of the data in this new space.

In this new space we can perform standard PCA, but with m (too high!) number of features. This is not possible for too high m (thousands).

Kernel trick

Fortunately, there is a kernel trick! In many machine learning methods, the dot product over the data is applied. The results of the dot product of two vectors is a scalar value. So, consider the dot product of two vectors (in 2 dimensions):

$$\mathbf{x}_i^T \mathbf{x}_j = \langle \mathbf{x}_i, \mathbf{x}_j \rangle = \left\langle \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1} \\ x_{j2} \end{bmatrix} \right\rangle = x_{i1} x_{j1} + x_{i2} x_{j2}$$

Now, consider the dot product after transforming the features into higher dimension (from previous example):

$$\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = \left\langle \begin{bmatrix} x_{i1}^2 \\ x_{i2}^2 \\ \sqrt{2}x_{i1} \cdot x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1}^2 \\ x_{j2}^2 \\ \sqrt{2}x_{j1} \cdot x_{j2} \end{bmatrix} \right\rangle = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1} x_{i2} x_{j1} x_{j2}$$

Kernel trick

The disadvantage of this approach is that we first need to transform all the feature set into high dimension. And because we typically have many features (not only two) it creates very high dimensional space ($p \gg m$) and therefore it is computationally demanding and also the memory storage increases.

Fortunately, we can do it in a smarter way:

$$\langle \mathbf{x}_i, \mathbf{x}_j \rangle^2 = \left\langle \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1} \\ x_{j2} \end{bmatrix} \right\rangle^2 =$$

$$(x_{i1}x_{j1} + x_{i2}x_{j2})^2 = x_{i1}^2x_{j1}^2 + x_{i2}^2x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

So, we are able to compute the dot product **without** explicitly going to higher dimension!

Kernel PCA

Now, let's apply this *kernel trick* to PCA. We must rewrite it in order to obtain the dot product in the eigendecomposition.

We start with covariance matrix of the projected features (size $p \times p$), calculated as

$$\mathbf{C} = \frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T$$

Its eigenvalues and eigenvectors are given by:

$$\mathbf{C} \mathbf{q}_k = \lambda_k \mathbf{q}_k,$$

for $k = 1, \dots, p$.

p is the number of features after transforming the features

We would like to do kernel PCA without having to compute $\mathbf{C}$ or $\phi(\mathbf{x})$. We will use kernel trick.

Kernel PCA

k - index for number of principal component.

Substitute the definition of C into last equation:

$$\frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i) [\phi(\mathbf{x}_i)^T \mathbf{q}_k] = \lambda_k \mathbf{q}_k.$$

The term in bracket gives a single number (multiplication of row and column vector) and dividing this number by λ_k and n gives new variable a_{ki} . It can be then written as:

$$\mathbf{q}_k = \sum_{i=1}^n a_{ki} \phi(\mathbf{x}_i).$$

—————→ The eigenvectors are linear combination of transformed features.

Now, substituting this equation into previous equation leads to:

$$\frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T \sum_{j=1}^n a_{kj} \phi(\mathbf{x}_j) = \lambda_k \sum_{i=1}^n a_{ki} \phi(\mathbf{x}_i)$$

and multiply both sides of the last equation by $\phi(\mathbf{x}_l)^T$, we have:

$$\frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_l)^T \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T \sum_{j=1}^n a_{kj} \phi(\mathbf{x}_j) = \lambda_k \sum_{i=1}^n a_{ki} \phi(\mathbf{x}_l)^T \phi(\mathbf{x}_i).$$

Kernel PCA

This is dot product between transformed samples - can be computed via kernel trick.

If we define the kernel function: $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$,

we can rewrite the last equation to

$$\frac{1}{n} \sum_{i=1}^n \kappa(\mathbf{x}_1, \mathbf{x}_i) \sum_{j=1}^n a_{kj} \kappa(\mathbf{x}_i, \mathbf{x}_j) = \lambda_k \sum_{i=1}^n a_{ki} \kappa(\mathbf{x}_1, \mathbf{x}_i).$$

Define K as a $n \times n$ kernel matrix:

$$K = \phi(\mathbf{X})\phi(\mathbf{X})^T =$$

$$\begin{bmatrix} \kappa(\mathbf{x}_1, \mathbf{x}_1) & \kappa(\mathbf{x}_1, \mathbf{x}_2) & \dots & \kappa(\mathbf{x}_1, \mathbf{x}_n) \\ \kappa(\mathbf{x}_2, \mathbf{x}_1) & \kappa(\mathbf{x}_2, \mathbf{x}_2) & \dots & \kappa(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa(\mathbf{x}_n, \mathbf{x}_1) & \kappa(\mathbf{x}_n, \mathbf{x}_2) & \dots & \kappa(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix}$$

The n -elements in each column of K represent the dot product of one point of the transformed data with respect to all the transformed points (n points).

Kernel function

Define $\mathbf{a}_k$ is the n -dimensional column vector: $\mathbf{a}_k =$

$$\begin{bmatrix} a_{k1} \\ a_{k2} \\ \vdots \\ a_{kn} \end{bmatrix}$$

With this matrix notation we can write:

$$\mathbf{K}^2 \mathbf{a}_k = \lambda_k n \mathbf{K} \mathbf{a}_k$$

and finally $\mathbf{a}_k$ and λ_k can be solved by:

$$\mathbf{K} \mathbf{a}_k = \lambda_k n \mathbf{a}_k,$$

which is similar to linear PCA, for which we had:

$$\mathbf{C} \mathbf{q}_j = \lambda_j \mathbf{q}_j, \quad j = 1, \dots, m.$$

λ_k are the eigenvalues, $\mathbf{a}_k$ are eigenvectors, both connected with covariance matrix $\mathbf{K}$.

Kernel PCA

In linear PCA, we the data must be centered before computing the covariance matrix.

For kernel PCA, we need to do the same, because the after application of kernel, the matrix is generally not centered.

In the matrix notation, the centered K is:

$$\tilde{\mathbf{K}} = \mathbf{K} - \mathbf{1}_n \mathbf{K} - \mathbf{K} \mathbf{1}_n + \mathbf{1}_n \mathbf{K} \mathbf{1}_n$$

Where $\mathbf{1}_n$ is $n \times n$ matrix with every element = $1/n$.

Eigen-decomposition is then done for the centered kernel matrix.

Summary of the algorithm

1. Construct the $n \times n$ kernel matrix $\mathbf{K}$.
2. Center $\mathbf{K}$.
3. Do eigendecomposition of centered $\mathbf{K}$ and find top L eigenvectors $\mathbf{a}_1, \dots, \mathbf{a}_L$ with eigenvalues $\lambda_1, \dots, \lambda_L$.
4. For any data point $\mathbf{x}$, we can represent it as the following set of features:

$$y_k = \sum_{i=1}^n a_{ki} \kappa(\mathbf{x}, \mathbf{x}_i)$$

for $k = 1, \dots, L$.

PCA, notes

- Kernel PCA can give a good re-mapping of the data when it lies along a non-linear “space”.
- The kernel matrix is $n \times n$, so kernel PCA will have difficulties if we have lots of data points.
- Results depends on the kernel.

Kernel	Equation
Linear	$K(x, y) = x \cdot y$
Sigmoid	$K(x, y) = \tanh(ax \cdot y + b)$
Polynomial	$K(x, y) = (1 + x \cdot y)^d$
KMOD	$K(x, y) = a \left[\exp\left(\frac{\gamma}{\ x - y\ ^2 + \sigma^2}\right) - 1 \right]$
RBF	$K(x, y) = \exp(-a\ x - y\ ^2)$
Exponential RBF	$K(x, y) = \exp(-a\ x - y\)$

Linear Discriminant Analysis

LDA

Linear discriminant analysis

In PCA, the class label of each and every example, even if available, is ignored and only the feature values from each example are considered. Thus, PCA is considered an unsupervised approach. The emphasis in PCA is to preserve data variability as much as possible while reducing the dimensionality.

Linear discriminant analysis (LDA) on the other hand makes use of class labels as well and its focus is on finding a lower dimensional space that emphasizes class separability, i.e. it seeks a projection that best separates the data in a least-squares sense (efficient for data discrimination).

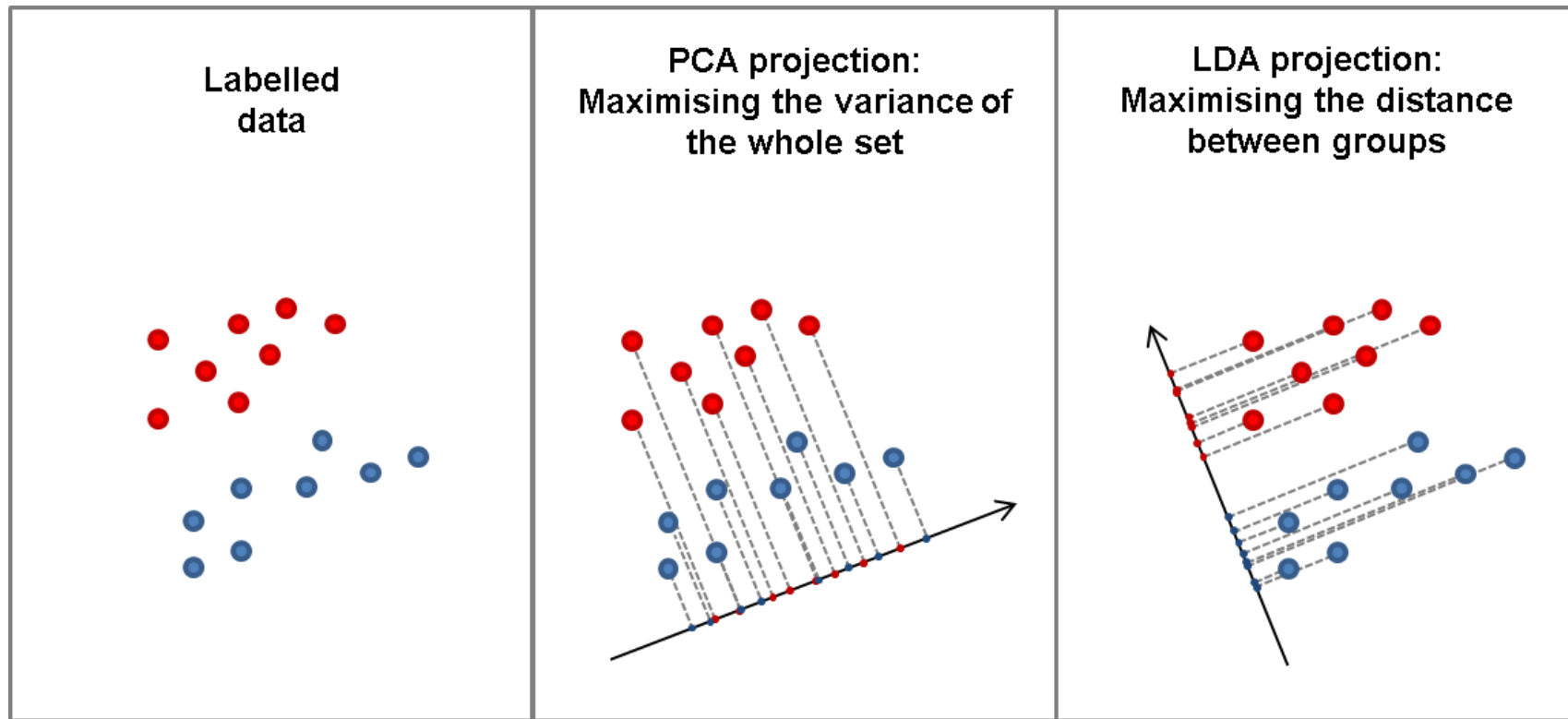
References:

<https://iksinc.online/2018/11/12/dimensionality-reduction-via-linear-discriminant-analysis/>

Book: Duda, R. et al. Pattern Classification, Second Edition, John Wiley & Sons, 2001, page 117-121

LDA - principle

The figure shows only two choices for the line onto which the projections are made. Since our goal is to prefer a projection where the projected data from two classes do not overlap as far as possible, clearly the projection on the right is preferable.



LDA

Given that an infinite number of projections are possible, we need to define a measure that can help us to select the best projection. Intuitively, we would like the projected points from two classes to be **as far apart as possible**. We can capture this notion by measuring the **difference between means** of class 1 and class 2 projected points. Although means can be far apart, it is still possible for projected points from two classes to show a fair amount of overlap if the projected points are spread out. To counter this, we also want the projected points from each class to lie **close to each other**. This means the mapped points for each class should have **as small a spread as possible**.

How to find the projection defined by vector $\mathbf{w}$?

LDA

Suppose a two-class problem with labeled training data. There are n_i samples in each class D_i ($i=1,2$). First, we define the *sample* mean feature vector for each class:

$$\mathbf{m}_i = \frac{1}{n_i} \sum_{x \in D_i} \mathbf{x}_i$$

Then the sample mean for projected points (y_i) is given by

$$\begin{aligned} \tilde{m}_i &= \frac{1}{n_i} \sum_{y \in Y_i} y_i = \\ &= \frac{1}{n_i} \sum_{x \in D_i} \mathbf{w}_i^T \mathbf{x} = \mathbf{w}_i^T \mathbf{m}_i \end{aligned}$$

and it is simply a projection of the mean vector onto $\mathbf{w}$. So we are able to compute the position of means for both classes.

LDA

The purpose of LDA is to **maximize distance** between these projected means:

$$|\tilde{\mathbf{m}}_1 - \tilde{\mathbf{m}}_2| = |\mathbf{w}^T (\mathbf{m}_1 - \mathbf{m}_2)|$$

We can make this difference as large as we wish by scaling $\mathbf{w}$. But, we also want to have a **small sample variance** (called here as a *scatter*) within the projected class, i.e.:

$$\tilde{\mathbf{s}}_i^2 = \sum_{y \in Y_i} (\mathbf{y} - \tilde{\mathbf{m}}_i)^2.$$

With the help of this scatter, we define the criterium (**Fisher linear discriminant**), which takes into account distance between projected means and also the total **within class scatter** $\tilde{s}_1^2 + \tilde{s}_2^2$:

$$J(\mathbf{w}) = \frac{|\tilde{\mathbf{m}}_1 - \tilde{\mathbf{m}}_2|^2}{\tilde{\mathbf{s}}_1^2 + \tilde{\mathbf{s}}_2^2}$$

LDA

To find the $\mathbf{w}$, we must rewrite $J(\mathbf{w})$ as an explicit function of $\mathbf{w}$. First, we define the scatter matrices $\mathbf{S}_i$:

$$\mathbf{S}_i = \sum_{x \in D_i} (\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^T$$

and

$$\mathbf{S}_w = \mathbf{S}_1 + \mathbf{S}_2.$$

Then we can write

$$\begin{aligned} \tilde{s}_i^2 &= \sum_{x \in D_i} (\mathbf{w}^T \mathbf{x} - \mathbf{w}^T \mathbf{m}_i)^2 = \\ &= \sum_{x \in D_i} \mathbf{w}^T (\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^T \mathbf{w} = \\ &= \mathbf{w}^T \mathbf{S}_i \mathbf{w} \end{aligned}$$

LDA

Therefore the sum of these scatters can be written as:

$$\tilde{s}_1^2 + \tilde{s}_2^2 = \mathbf{w}^T \mathbf{S}_w \mathbf{w}$$

Similarly, the separation of projected means:

$$\begin{aligned} (\tilde{m}_1 - \tilde{m}_2)^2 &= (\mathbf{w}^T \mathbf{m}_1 - \mathbf{w}^T \mathbf{m}_2)^2 = \\ &\mathbf{w}^T (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T \mathbf{w} = \\ &\mathbf{w}^T \mathbf{S}_B \mathbf{w}, \end{aligned}$$

where

$$\mathbf{S}_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T$$

LDA

The final expression for J is:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}},$$

where $\mathbf{S}_w$ is called within-class scatter matrix and $\mathbf{S}_B$ is between-class scatter matrix.

Note, that $\mathbf{S}_B$ is computed only from vector of mean values ($(\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T$) and $\mathbf{S}_w$ is computed from all features $((\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^T)$.

The solution $\mathbf{J}(\mathbf{w})$ can be obtained (without proof) by differentiation with respect to $\mathbf{w}$ and we get:

$$\mathbf{w} = \mathbf{S}_w^{-1} (\mathbf{m}_1 - \mathbf{m}_2)$$

Remark: $J(\mathbf{w})$ is also known as generalized Rayleigh quotient.

LDA

The last term is known as Fisher's linear discriminant (1936), although it is not a discriminant but rather a specific choice of direction for the projection of the data down to one dimension. **So we have mapping “many-to-one” dimension.**

We found an optimal projection of our data onto 1D space. The last step is missing to obtain the classifier - to find the threshold for projected data. This will be shown later in “Linear classifier” part of this course.

There is also similar explanation for **multiclass** LDA.

LDA and eigenvectors

What can be proofed is a rather general formula:

$$[\mathbf{S}_w^{-1}\mathbf{S}_B]\mathbf{W} = \Lambda\mathbf{W}$$

Actually, this is a definition of eigenvalues and eigenvectors of matrix $\mathbf{X}$:

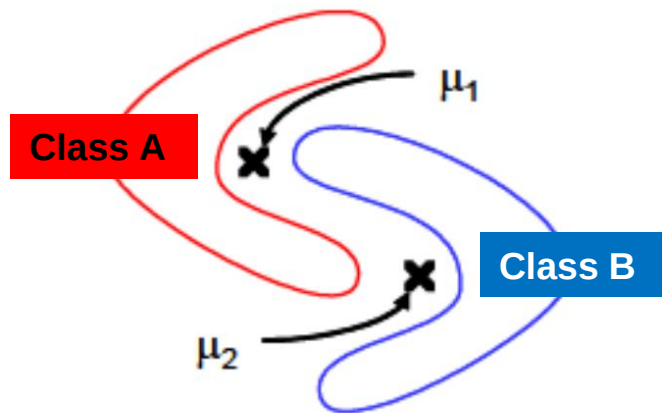
$$\mathbf{X}\mathbf{W} = \Lambda\mathbf{W},$$

where Λ is a diagonal matrix of eigenvalues and $\mathbf{W}$ is a matrix of eigenvectors.

Thus computing eigen-decomposition of $[\mathbf{S}_w^{-1}\mathbf{S}_B]$, leads to orthogonal projection to more dimensions according to LDA principle.

Limitations of LDA

- LDA is a *parametric method* since it assumes unimodal Gaussian feature distribution - if the distributions are significantly non-Gaussian, the LDA projections will not be able to preserve any complex structure of the data, which may be needed for classification.



t - SNE

(Stochastic Neighbor Embedding)

t-SNE

SNE technique was presented in 2002 and (now very popular technique) t-SNE presented in 2008.

Main author:

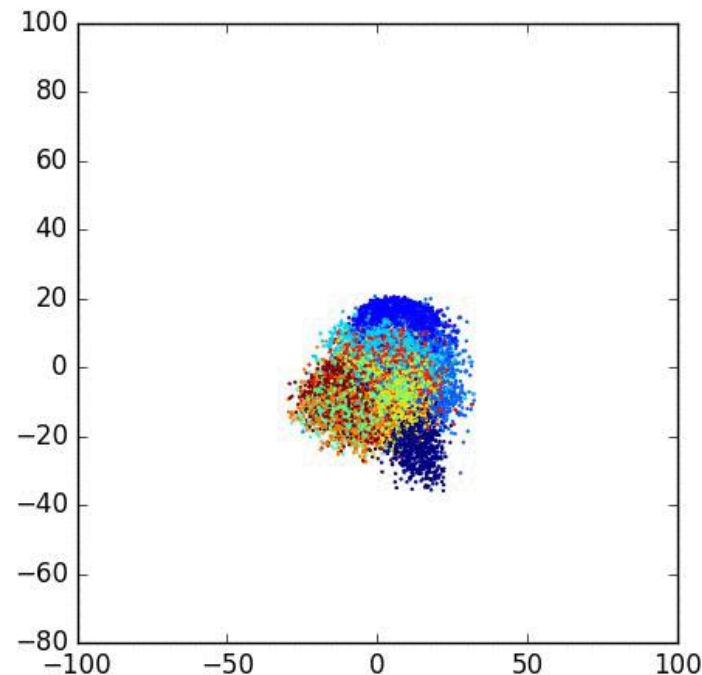
Laurens van der Maaten

Many implementations:

<https://lvdmaaten.github.io/tsne/>

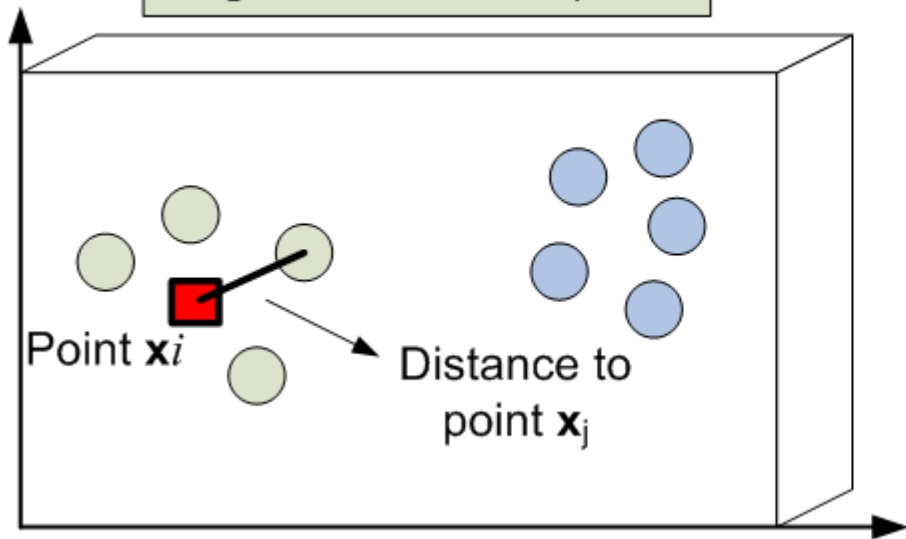
Simple explanation:

<https://www.youtube.com/watch?v=NEaUSP4YerM>

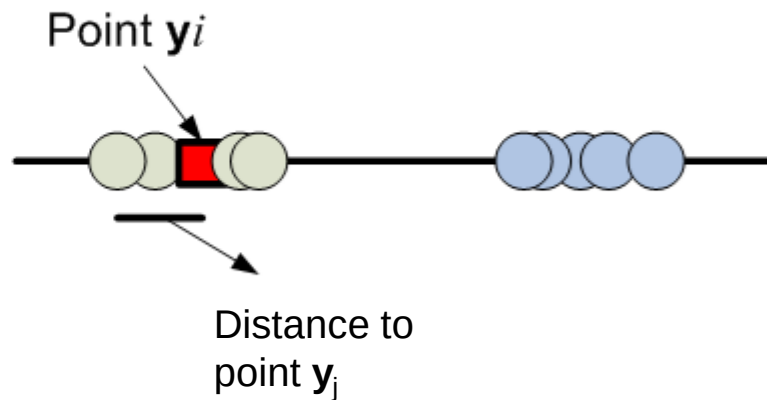


t-SNE, principle

High dimensional space



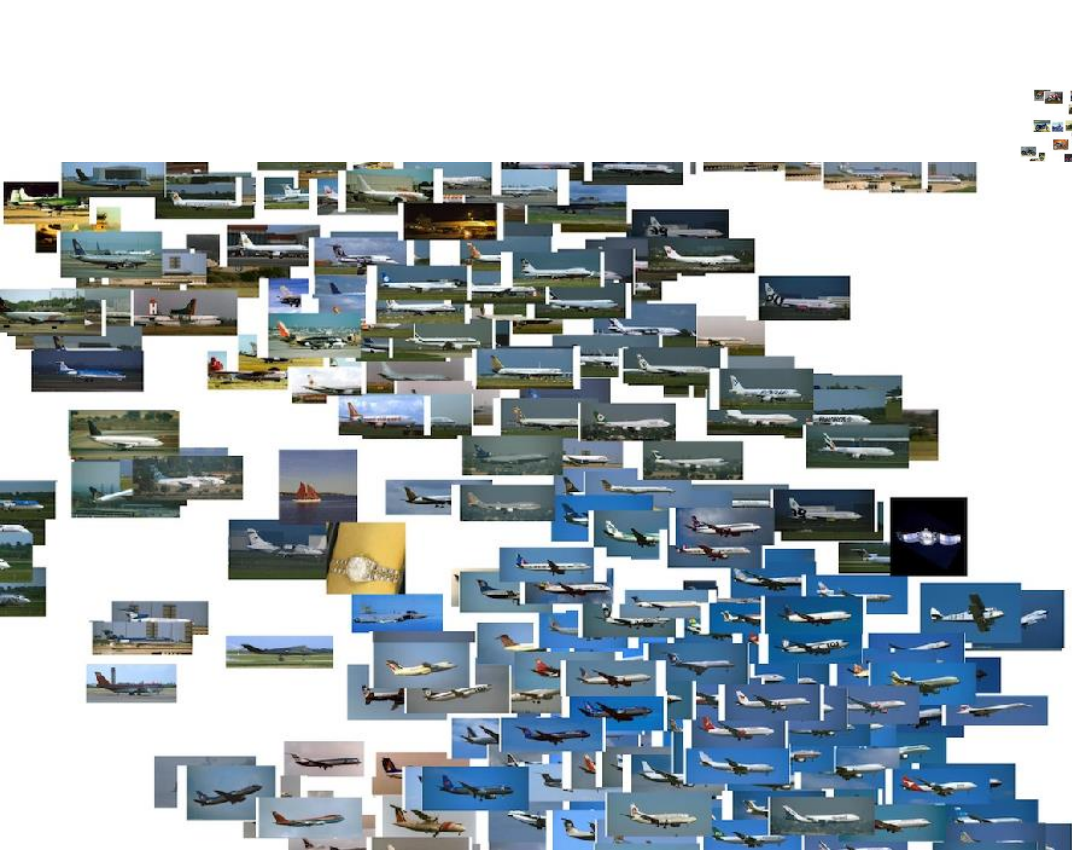
Low dimensional space



t-SNE

The aim of this techniques (and also other techniques) is to convert datapoints from high dimensional space into low dimension (typically 2) while preserving as much of the significant **structure** of the high-dimensional data as possible in the low-dimensional space. In the high dimensional space we cannot directly observe (i.e. see) this local structure of datapoints. But in 2D or 3D it is much easier. This method is usually considered as a visualization technique rather than feature reduction technique. But can be used for both purposes.

Example: <https://www.auduno.com/2018/10/27/visualizing-an-art-collection/>



t-SNE

Stochastic Neighbor Embedding (SNE) starts by converting the high-dimensional Euclidean distances between datapoints into similarities (sometimes called as *conditional probabilities*). The **similarity** $p_{j|i}$ of datapoint $\mathbf{x}_j$ to datapoint $\mathbf{x}_i$ is the probability, that $\mathbf{x}_i$ would pick $\mathbf{x}_j$ as its neighbor if neighbors were picked in proportion to their probability density under a Gaussian centered at $\mathbf{x}_i$. For nearby datapoints, $p_{j|i}$ is relatively high, whereas for widely separated datapoints, $p_{j|i}$ will be almost infinitesimal (for reasonable values of the variance of the Gaussian, σ_i).

Mathematically, this similarity $p_{j|i}$ is given by

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)},$$

where σ_i is the variance of the Gaussian that is centered on datapoint $\mathbf{x}_i$.

t-SNE

Variance of the Gaussian is a *free* parameter that significantly influences the result. The Gaussian is centered over each high-dimensional datapoint, $\mathbf{x}_i$. It is not likely that there is a single value of σ_i that is optimal for all datapoints in the dataset because the density of the data is likely to vary. **In dense regions, a smaller value of σ_i is usually more appropriate than in sparser regions.** Any particular value of σ_i induces a probability distribution, P_i , over all of the other datapoints. This distribution has an entropy which increases as σ_i increases. t-SNE performs a [binary search](#) for the value of σ_i that produces a P_i with a fixed [perplexity](#) that is specified by the user. The perplexity is defined for each point as

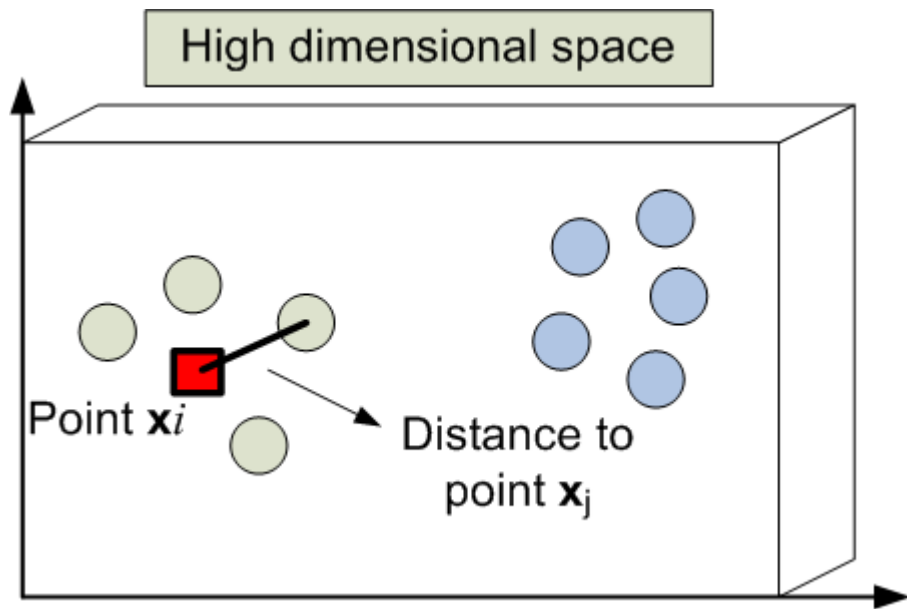
$$Perp(P_i) = 2^{H(P_i)},$$

where $H(P_i)$ is the Shannon entropy of P_i measured in bits $H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$.

t-SNE

The perplexity can be interpreted as a smooth measure of the **effective number of neighbors**. The performance of SNE is fairly robust to changes in the perplexity, and typical values are between 5 and 50.

t-SNE



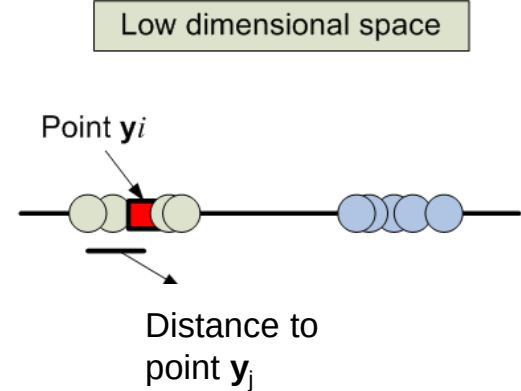
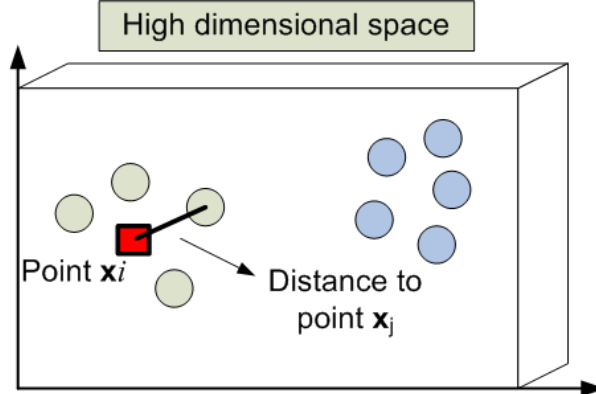
$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

The matrix of these conditional probabilities is not symmetric mainly due to different values of sigma for particular point $\mathbf{x}_i$, which cause problems in mapping and therefore a symmetric version is used:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

where n is a number of datapoints.

t-SNE



We would like to represent/visualize this data in low dimensional space (e.g. 2D). Now consider that each datapoint in high-D ($\mathbf{x}_i$) has its own “image” in low-D ($\mathbf{y}_i$) and we can also compute the conditional probabilities for each point $\mathbf{y}_i$. It turned out that it is better to use slightly different model, Student t-distribution:

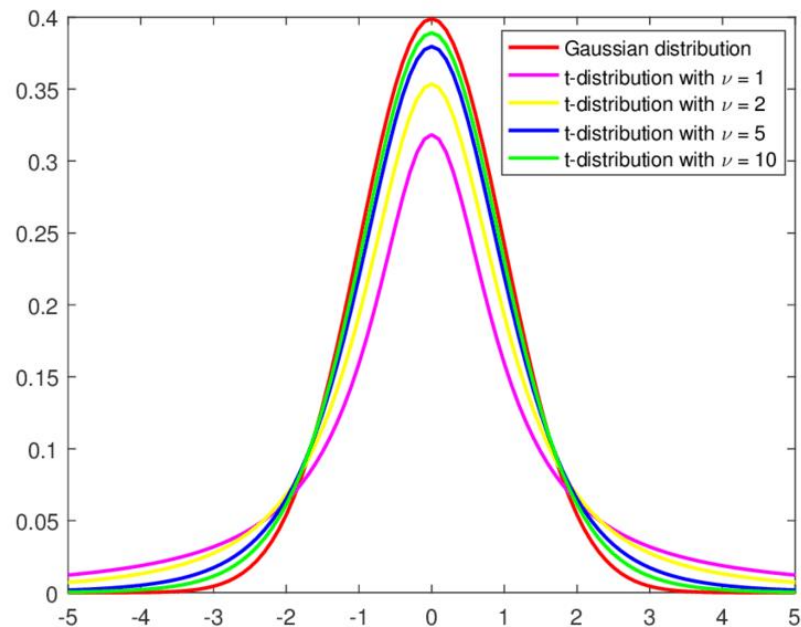
$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

This distribution has much heavier tails than Gaussian. This allows a moderate distance in the high-dimensional space to be faithfully modeled by a much larger distance in the low-dimensional space.

t-SNE

This distribution is a [Student distribution](#) with degree of freedom equal to 1. It therefore has no other parameter to be set.

Due to heavier tails, it enables to distribute the points in low-dimensional space into large distance and model the distances more accurate. This helps to avoid to the “crowding problem” - it is impossible to model *all* the distances between datapoints in high - dimensional (e.g. 100) space also in low-dimensional (e.g. 2D) space.



https://www.researchgate.net/publication/321085266_Gaussian_process_regression_methods_and_extensions_for_stock_market_prediction

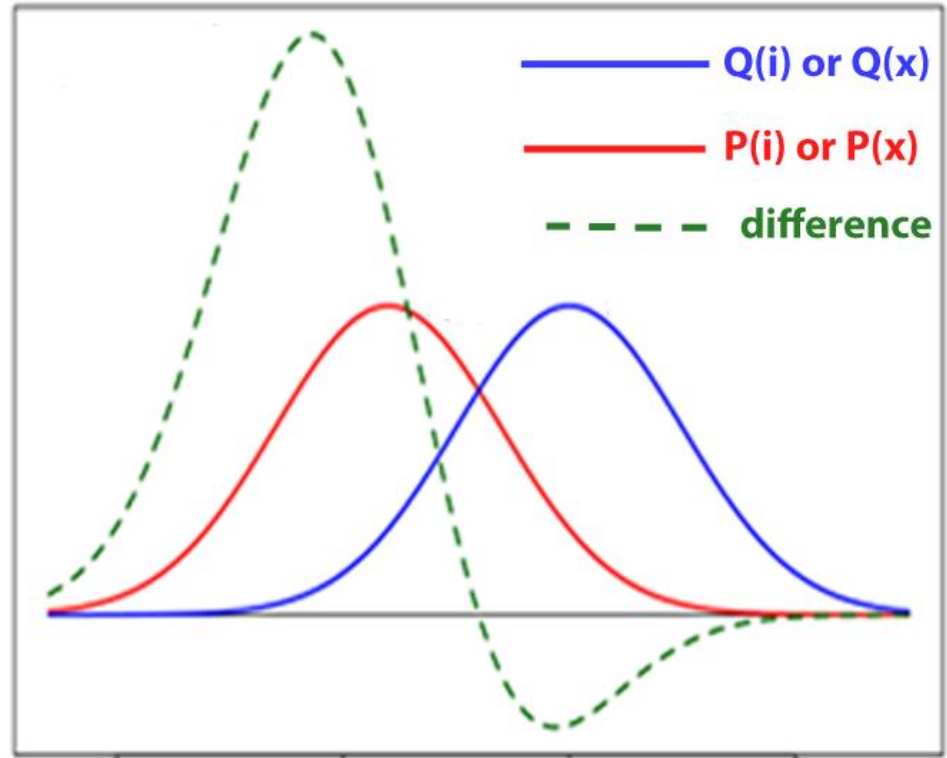
t-SNE

Now we've somehow represented the data in high- and low-dimensional space using these probabilities, p_{ij} and q_{ij} . If the points $\mathbf{y}_i$ and $\mathbf{y}_j$ correctly model the similarity between the high-dimensional datapoints $\mathbf{x}_i$ and $\mathbf{x}_j$, the conditional probabilities p_{ji} and q_{ji} will be equal. Motivated by this observation, t-SNE aims to find a low-dimensional data representation that minimizes the mismatch between p_{ij} and q_{ij} . A natural measure of the faithfulness with which q_{ij} models p_{ij} is the **Kullback-Leibler divergence**. t-SNE minimizes the sum of Kullback-Leibler divergences over all datapoints (with respect to new datapoints $\mathbf{y}_i$) using a gradient descent method. The cost function C is given by

$$C = KL(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Kullback-Leiber divergence

The Kullback–Leibler divergence (also called relative entropy) is a measure of how one probability distribution is different from a second, reference probability distribution.



$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

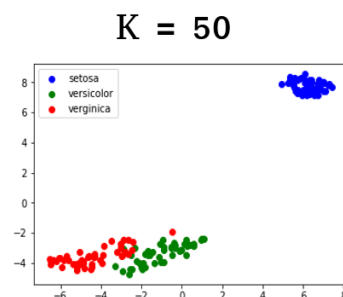
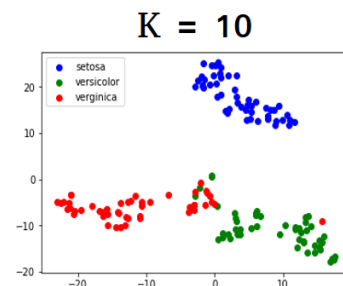
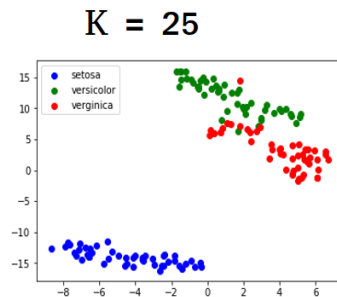
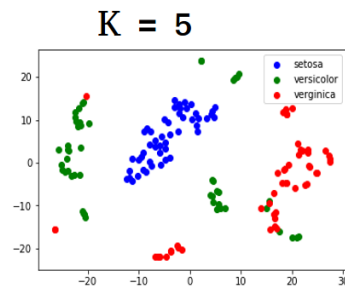
$$D_{KL}(P||Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$$

t-SNE properties

The input features are converted to output features (often 2D or 3D space). However, after this process, the input features are no longer identifiable.

Mainly used for data exploration and visualization of high-dimensional dataspace.

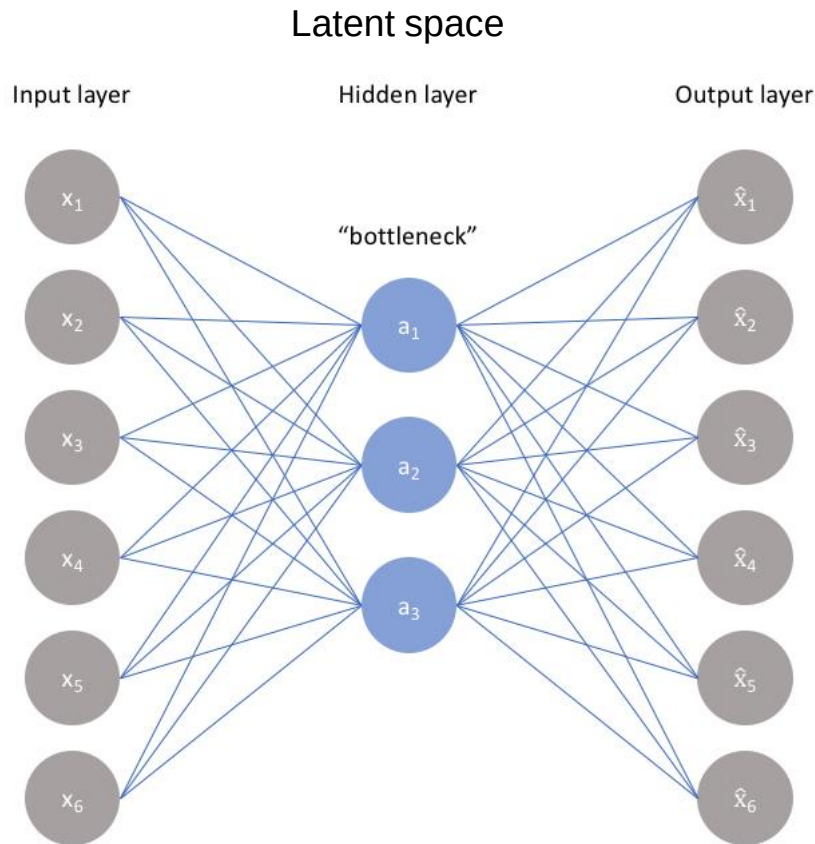
Discovering patterns using this visualization at an early stage of your machine learning pipeline, helps to guide the next steps of data analysis. If categories are well-separated by t-SNE, *machine learning* is likely to be able to find a good classification.



Autoencoders

Autoencoders

Another popular dimensionality reduction method are autoencoders, a type of artificial neural network that aims to **copy their inputs to their outputs**. They compress the input into a latent-space representation, and then reconstructs the output from this representation.



Autoencoders

There are three main parts of autoencoder:

Encoder: compresses the input into a latent-space representation.

Decoder: reconstruct the input from the latent space representation.

Bottleneck: constrains the amount of information that can traverse the full network, forcing a learned compression of the input data.

We can take an unlabeled dataset and learn this neural network (with non-linear activation functions of each neuron) by providing the $\mathbf{x}$ at the input and ask for the same output $\hat{\mathbf{x}}$. Therefore, this network can be trained by minimizing the reconstruction error, $L(\mathbf{x}, \hat{\mathbf{x}})$, which measures the differences between our original input and the consequent reconstruction. The bottleneck is a key property - without this bottleneck, our network could easily learn to simply memorize the input values by passing these values along through the network.

Autoencoders

Different architecture of autoencoders (basic, sparse, undercomplete) and also different loss function, which are optimized during training.

To conclude: An autoencoder is a neural network architecture capable of discovering structure within data in order to develop a compressed representation of the input. Many different variants of the general autoencoder architecture exist with the goal of ensuring that the compressed representation represents meaningful attributes of the original data input; typically the biggest challenge when working with autoencoders is getting your model to actually learn a meaningful and generalizable latent space representation.

Common nonlinear dimensionality reduction techniques

- 1) Sammon mapping (1969)
- 2) Curvilinear components analysis (1997)
- 3) Stochastic Neighbor Embedding (2002), t-SNE (2008)
- 4) Isomap (2000)
- 5) Maximum Variance Unfolding (2004)
- 6) Locally Linear Embedding (2000)
- 7) Laplacian Eigenmaps (2002)
- 8) Kernel PCA
- 9) Autoencoders

Feature selection methods
(random tree, forward/backward, MRMR)

Feature selection methods

The advantage of feature selection - it keeps the original features, thus making the interpretation easy.

Problem statement

Suppose we have a set of features F with features x_i ; $i = 1; 2; \dots; P$. We want to select only *the best* features according to some criterium.

But:

What is the best criterium?

Feature selection based on random
tree classifier

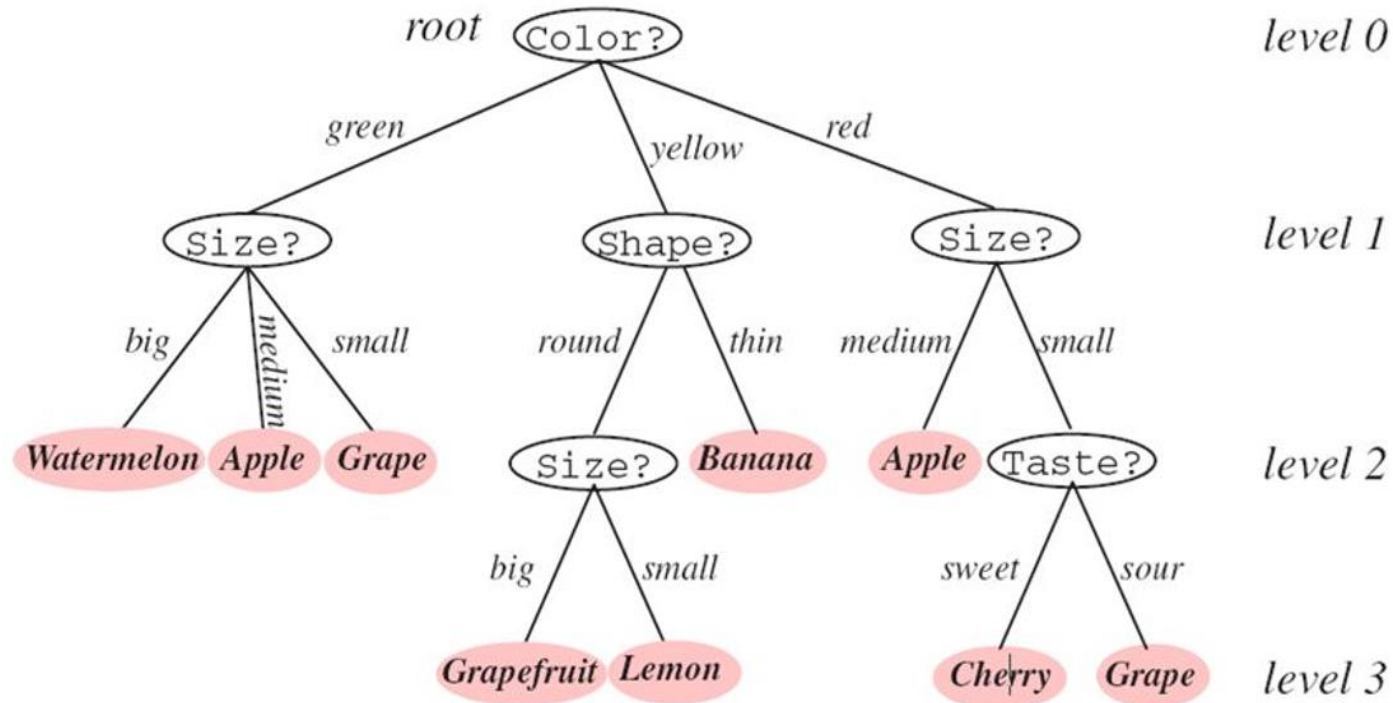
Feature selection based on Random Tree

Random forests are among the most popular machine learning methods thanks to their relatively good accuracy, robustness and ease of use. They also provide two straightforward methods for feature selection: mean decrease impurity and mean decrease accuracy.

More details about random trees and forest will be given in a separate lecture.

Here, just the main idea on next slides.

Feature selection based on Random Tree



Feature selection based on Random Tree

1. Prepare the dataset
2. Train a random forest classifier
3. Identify the most important features
4. Create a new 'limited featured' dataset containing only those features
5. Train a second classifier on this new dataset
6. Compare the accuracy of the 'full featured' classifier to the accuracy of the 'limited featured' classifier

Forward feature selection

Forward selection

The forward selection tries to select a subset of M features (we denote the subset by Z) from whole feature set in an incremental manner.

The wrapper first searches for the feature subset with one feature x_1 ($Z_1=\{x_1\}$) that leads to the largest error reduction.

Then, from the set $\{S_n - x_1\}$, the wrapper selects the feature x_2 so that the feature set $Z_2=\{x_1, x_2\}$ leads to the largest error reduction.

This incremental selection repeats until the classification error begins to increase, i.e., $e_{k+1} > e_k$. Once the termination condition is satisfied, the selected number of features, M , is chosen as the dimension for which the lowest error is first reached.

Backward feature selection

Backward feature selection

The backward selection tries to exclude one redundant feature at a time from the current feature set S_k (initially, S_k is set to F), with the constraint that the resultant feature set S_{k-1} leads to a classification error e_{k-1} **no worse** than e_k . Because every feature in S_k can be considered in removal, there are k different configurations of S_{k-1} . For each possible configuration, the respective classification error e_{k-1} is calculated. Among the k configurations of S_{k-1} , the one that leads to the largest error reduction is chosen as the new feature set. If, for every configuration, the corresponding error e_{k-1} is larger than e_k , there is no gain in either classification accuracy or feature dimension reduction (i.e. every existing feature in S_k is useful), thus, the backward selection terminates. If there are multiple configurations leading to the same error reduction, one of them is chosen randomly.

Maximum relevance
minimum redundancy
(MRMR)

Maximum relevance, minimum redundancy (MRMR)

Suppose a feature space F with features x_i ; $i = 1; 2; \dots; P$ and target classes (labels) c . Furthermore suppose its subset $S \subseteq F$. The aim of features selection method is to find S with $M < P$ features x_i ; $i = 1; 2; \dots; M$ according to defined criteria.

Max-Relevance and Min-Redundancy scheme is based on two concepts: a good feature should have **maximum relevance to target class c** and **minimum redundancy to already selected features**.

These two properties can be described by mutual information.

Mutual information

The mutual information I is a measure of the amount of information that one random variable has about another variable:

$$I(x; y) = \sum_{i=1}^n \sum_{j=1}^n p(x(i), y(j)) \cdot \log \left(\frac{p(x(i), y(j))}{p(x(i)) \cdot p(y(j))} \right)$$

where MI is zero when x and y are statistically independent: $p(x(i), y(i)) = p(x) \cdot p(y)$.

The MI can be expressed as:

$$I(x; y) = H(x) + H(y) - H(x, y)$$

Max - Relevance, MRMR - cont'd

The Max-Relevancy scheme selects such features x_i , which have largest mutual information $I(x_i; c)$ with the target class c . The features are selected in such a way that the mean value D of selected mutual information is maximized:

$$\max_S D(S, c); \quad D(S, c) = \frac{1}{P} \sum_{x_i \in S} I(x_i, c)$$

Min-Redundancy - MRMR

The Min-Redundancy scheme selects features according to their redundancies. When two features are highly dependent on each other, their selection does not change so much the respective classification accuracy. Therefore, this scheme for the features selection is based on the minimization of function $R(S)$:

$$\min_S R(S); \quad R(S) = \frac{1}{P} \sum_{x_i, x_j \in S} I(x_i, x_j)$$

MRMR

The MRMR is a combination of these two constraints. The final criteria can be constructed using difference:

$$\max_S (D(S) - R(S))$$

Or ratio:

$$\max_S \frac{D(S)}{R(S)}$$

MRMR

The features selection works in an iterative manner. First, the feature with the highest mutual information to the classes is selected (Max Relevancy). The rest features are selected incrementally, e.g. previously selected features stay in a subset S_{i-1} and the new feature x_i is selected in a such way, that it maximizes the final criterion to create $S_i = S_{i-1} + x_i$.

The basic paper introducing this method:

<https://ieeexplore.ieee.org/document/1453511>

Implemenation and online MRMR:

<http://home.penglab.com/proj/mRMR/>